



河南大學
HENAN UNIVERSITY

计算机操作系统

主讲人：胡 萍

明德，新民

止于至善

第二章 进程的描述与控制

第1讲



本章内容

2.1 前趋图和程序执行

2.2 进程的描述

2.3 进程控制

2.4 进程同步

2.5 经典进程的同步问题

2.6 进程通信

2.7 线程的基本概念

2.8 线程的实现





2.1 前趋图和程序执行

- 2.1.1 前趋图

- 2.1.2 程序顺序执行

- 2.1.3 程序并发执行



2.1.1 前趋图

□ 前趋图：一个有向无循环图，可记为DAG（Directed Acyclic Graph），用于描述进程之间执行的先后顺序。

■ 每个结点用来表示一个进程或程序段，乃至一条语句。

结点间有向边则表示两个结点之间的偏序或前驱关系。

■ 前驱关系： $\rightarrow$

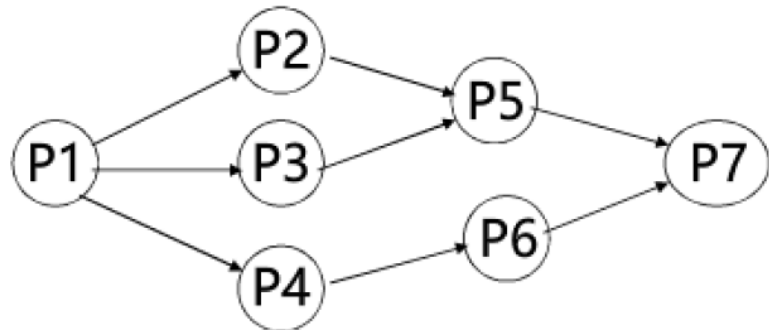
■ P_i 和 P_j 存在着前驱关系： $(P_i, P_j) \in \rightarrow$ 或者 $P_i \rightarrow P_j$

■ P_i 是 P_j 的直接前驱， P_j 是 P_i 的直接后继。



2.1.1 前趋图

- 初始结点：没有前驱的结点
- 终止结点：没有后继的结点
- 重量：每个节点所含有的程序量或程序的执行时间
- $P_1 \rightarrow P_2, P_1 \rightarrow P_3, P_1 \rightarrow P_4, P_2 \rightarrow P_5, P_3 \rightarrow P_5, P_4 \rightarrow P_6, P_5 \rightarrow P_7, P_6 \rightarrow P_7$
- $P = \{P_1, P_2, P_3, P_4, P_5, P_6, P_7\}$
 $= \{(P_1, P_2), (P_1, P_3), (P_1, P_4), (P_2, P_5), (P_3, P_5), (P_4, P_6), (P_5, P_7), (P_6, P_7)\}$



前趋图中不允许有循环，否则会产生不可能实现的前驱关系



2.1.2 程序顺序执行

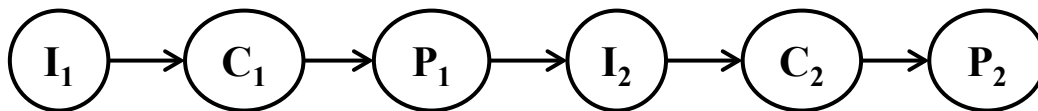
- 1. 程序的顺序执行
- 2. 程序顺序执行的特征



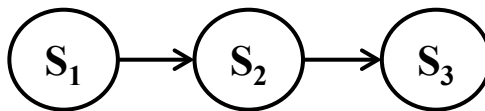
1. 程序的顺序执行

程序的顺序执行包含两层含义：

- 外部顺序性：对多个用户程序，所有程序是依次执行的。
- 内部顺序性：对一个程序，其所有指令是按序执行的。



S1: $a=x+y$
S2: $b=a-5$
S3: $c=b+1$





2. 程序顺序执行的特征

(1) 顺序性

处理机的操作严格按照程序所规定的顺序执行，即下一操作必须在当前操作结束后才能开始。

(2) 封闭性

程序是在封闭的环境下执行的。即

- 程序运行时独占全机资源，资源的状态（除初始态外）只有本程序才能改变它。
- 程序一旦开始执行，其执行结果不受外界影响

(3) 可再现性

只要程序执行时的环境和初始条件相同，当程序重复执行时，都将获得相同的结果。



2.1.3 程序并发执行

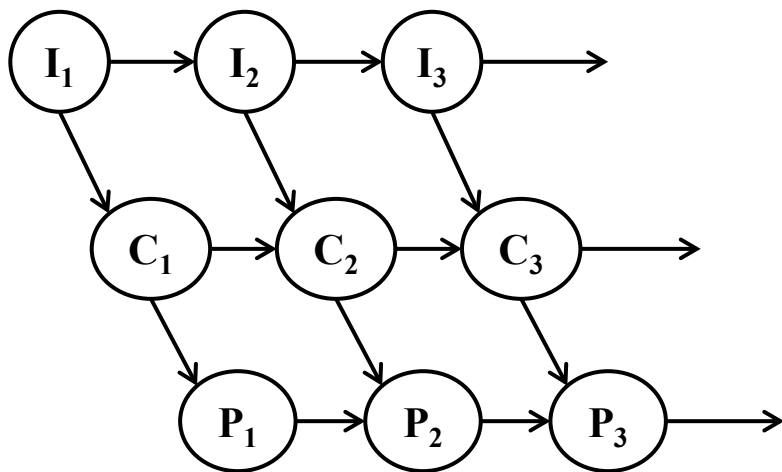
- 1. 程序的并发执行
- 2. 程序并发执行时的特征



1. 程序的并发执行

程序的并发执行包括两层含义：

- **外部并发性：**对于多个程序（进程）来说，所有进程是交叉执行的。
- **内部顺序性：**对一个程序，其所有指令是按序执行的。

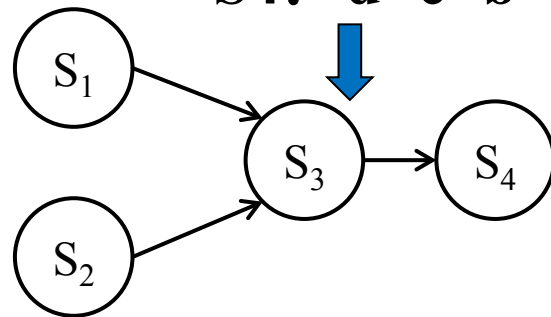


S1: $a=x+2$

S2: $b=y+4$

S3: $c=a+b$

S4: $d=c+b$

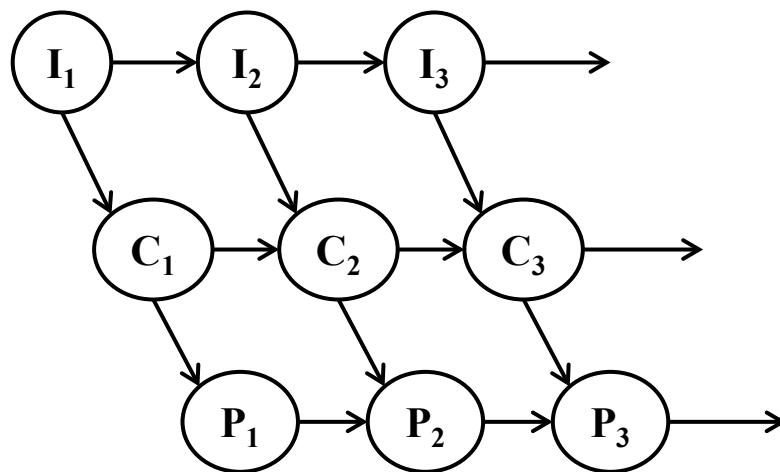




2. 程序并发执行时的特征

(1) 间断性：程序在并发执行时，由于它们共享系统资源，以及为完成同一任务而相互合作，致使这些并发执行的程序之间形成了相互制约的关系。（互斥关系、同步关系）

相互制约导致并发执行的程序具有“执行--暂停--执行”这种间断性活动规律。





2. 程序并发执行时的特征

(2) 失去封闭性：程序在并发执行时，由于多个程序共享系统资源，因而这些资源的状态将由多个程序来改变，致使程序的运行已失去了封闭性。



某程序的执行时，会受到其他程序的影响。



2. 程序并发执行时的特征

(3) 不可再现性：程序在并发执行时，由于失去了封闭性，也将导致其再失去可再现性。

例如：有两个循环程序A和B，它们共享一个变量N。

程序
A

L1: $N = N + 1$;
goto L1;

程序
B

L2: print (N);
N = 0;
goto L2;

程序A和B并发执行时，可能出现下述三种情况（设某时刻N的值为10）：

- (1) $N = N + 1$ 在print(N)和 $N = 0$ 之前，此时得到的N值分别为11，11，0
- (2) $N = N + 1$ 在print(N)和 $N = 0$ 之后，此时得到的N值分别为10，0，1
- (3) $N = N + 1$ 在print(N)和 $N = 0$ 之间，此时得到的N值分别为10，11，0

计算结果已与并发程序的执行速度有关，从而使程序执行失去了可再现性。



多道程序环境下，程序的执行属于**并发**执行，此时它们将失去其封闭性，并具有间断性和运行结果的不可再现性特征。

此时，通常的程序是不能参与并发执行的，否则，程序的运行也就失去了意义。

为了能使程序并发执行，并且可以对并发执行的程序加以描述和控制，引入了“进程”的概念

1. 程序的顺序执行通常在 [填 A 1] 的工作环境中，具有 [填 C 2] 特征；程序的并发执行在 [填 B 3] 的工作环境中，具有 [填 D 4] 特征。

A

单道程序

B

多道程序

C

程序的可再现性

D

资源共享

作答





2.2 进程的描述

- 2.2.1 进程的定义和特征
- 2.2.2 进程的基本状态及转换
- 2.2.3 挂起操作和进程状态的转换
- 2.2.4 进程管理中的数据结构

进程控制



2.2.1 进程的定义和特征

□ 1. 进程的定义

□ 2. 进程的特征



1. 进程的定义

□ 进程（**Process**）：是具有独立功能的程序在一个数据集合上运行的过程，是系统进行**资源分配和调度**的一个**独立单位**。

□ 几个要点：

- 进程是**程序**的一次执行；
- 进程一个程序及其数据在处理机上顺序执行时所发生的**活动**；
- 进程是程序在一个**数据集合**上运行的过程；
- 进程是系统进行**资源分配和调度**的一个独立单位。



1. 进程的定义

□ 进程结构：程序段、相关的数据段、进程控制块（PCB）三部分构成了进程实体。

- 进程控制块（PCB）：一个专门的数据结构，进程唯一标识；
- 程序段：进程运行过程中所使用的指令，可被多个进程共享；
- 数据段：进程运行过程中的相关数据（原始数据、中间数据）



2. 进程的特征

- (1) 动态性：进程的实质是进程实体的一次执行过程，故动态性是进程的**最基本特征**。
- (2) 并发性：指多个进程实体同存于内存中，且能在一段时间内同时运行。
- (3) 独立性：指**进程实体**是一个能独立运行、独立分配资源、独立接受调度的**基本单位**。
- (4) 异步性：是指进程按各自独立的、不可预知的速度向前推进，或说进程实体按**异步方式**运行。

配置相应的进程同步控制

2. 进程主要由 **程序段** **数据段** **PCB** 三部分内容组成，其中 **PCB** 是进程存在的唯一标志。

作答



3. 进程的基本特征有 **动态**、**并发**、独立、异步。

作答





2.2.2 进程的基本状态及转换

- 1. 进程的三种基本状态
- 2. 进程的三种基本状态的转换
- 3. 创建状态和终止状态



1. 进程的三种基本状态

处于就绪状态的进程可能有多个，通常将它们排成一个队列，称为**就绪队列**。

(1) 就绪 (Ready) 状态

当进程已分配到除CPU以外的所有资源后，只要再获得CPU，便可立即执行，进程这时的状态称为就绪状态。

(2) 执行 (Running) 状态

进程已获得CPU，其程序正在执行。

使进程阻塞的典型事件：
请求I/O、申请缓冲空间等

(3) 阻塞 (Blocked) 状态

正在执行的进程由于发生某事件而暂时无法继续执行时，便放弃处理机而处于暂停状态，即进程的执行受到阻塞，也称为等待状态。



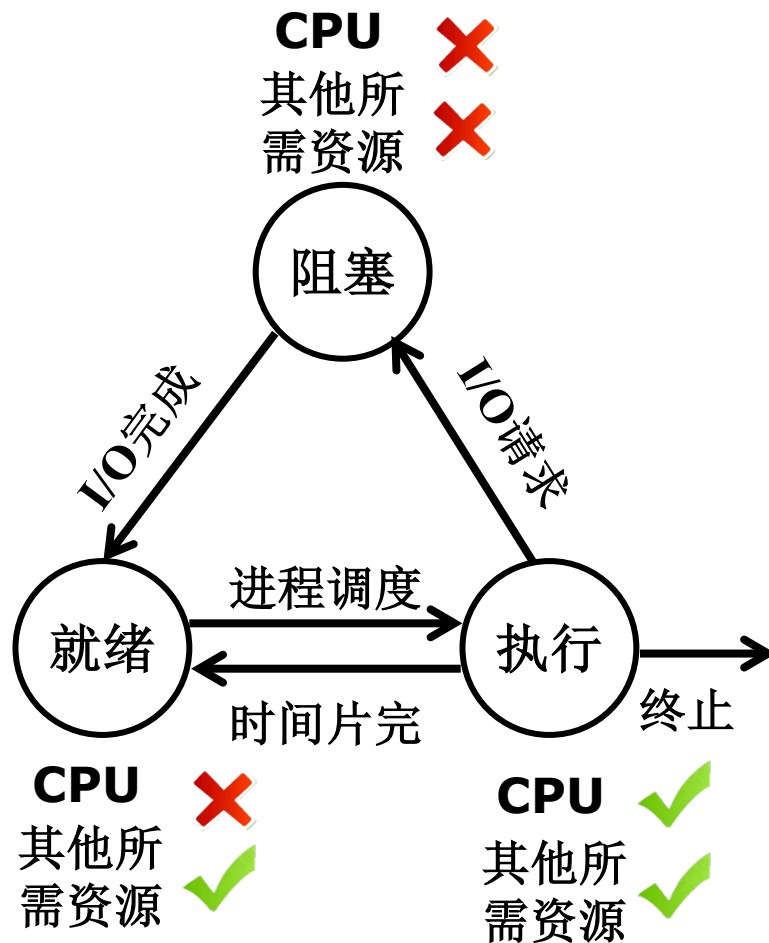
2. 进程的三种基本状态的转换

引起进程状态转换的典型事件：

- 就绪状态 $\xrightarrow{\text{进程调度}}$ 执行状态
- 执行状态 $\xrightarrow{\text{时间片完}}$ 就绪状态
- 执行状态 $\xrightarrow{\text{I/O请求}}$ 阻塞状态
- 阻塞状态 $\xrightarrow{\text{I/O完成}}$ 就绪状态



哪些状态转换不可能？

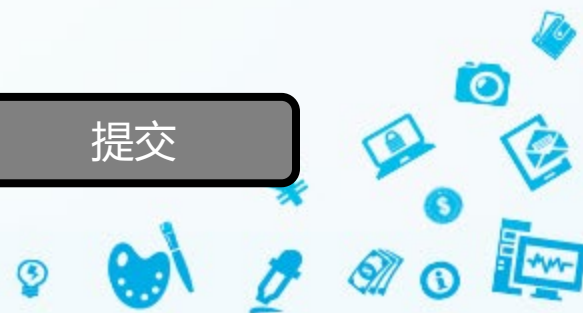


4. 张三正在抄作业——张三是处理机，“抄作业”操作是一个进程。当前“抄作业”进程处于什么状态？

- ☐ A 就绪
- ☒ B 执行
- ☐ C 阻塞
- ☐ D 都不是

B

提交



5. 张三正在抄作业——张三是处理机，“抄作业”操作是一个进程。忽然外卖到了，需要张三立即去取外卖——张三要暂停抄作业，转去执行“取外卖”进程。
此时“抄作业”进程处于什么状态？

A

- ☒ A 就绪
- ☐ B 执行
- ☐ C 阻塞
- ☐ D 都不是

提交



6. 张三正在抄作业——张三是处理机，“抄作业”操作是一个进程。忽然外卖到了，需要张三立即去取外卖——张三要暂停抄作业，转去执行“取外卖”进程。回来继续抄作业，忽然被抄作业的主人说有道题做错了，他需要拿回去修改。此时“抄作业”进程处于什么状态？

C

- ☐ A 就绪
- ☐ B 执行
- ☒ C 阻塞
- ☐ D 都不是

提交

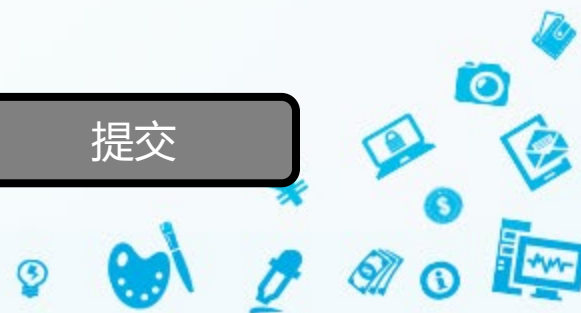


7. 在进程管理中，当（ ）时，进程从阻塞状态变为就绪状态。

C

- ☐ A 进程被进程调度程序选中
- ☐ B 等待某一事件
- ☒ C 等待的事件已发生
- ☐ D 时间片用完

提交

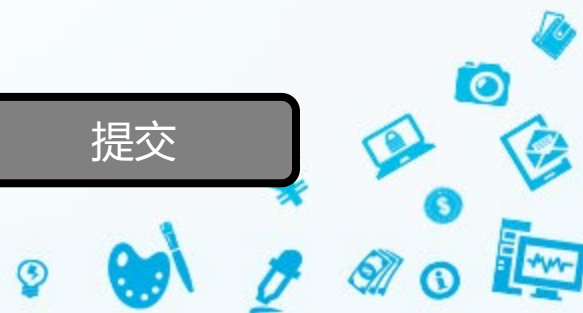


8. 分配到必要的资源并获得处理机时的状态是（ ）

B

- ☐ A 就绪状态
- ☒ B 执行状态
- ☐ C 阻塞状态
- ☐ D 撤消状态

提交



9. 进程的三个基本状态在一定条件下可以相互转化，进程由就绪状态变为运行状态的条件是（ ）

D

- ☐ A 时间片用完
- ☐ B 等待某事件发生
- ☐ C 等待的某事件已发生
- ☒ D 被进程调度程序选中

提交

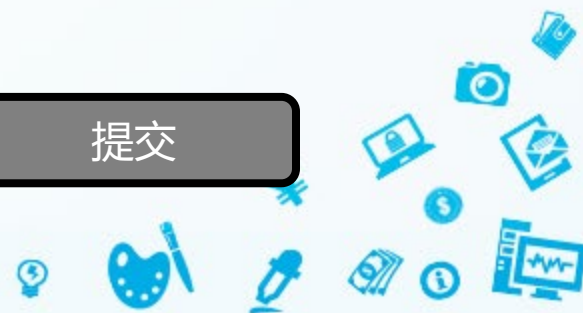


10. 进程的三个基本状态在一定条件下可以相互转化，进程由运行状态变为阻塞状态的条件是（ ）

B

- ☐ A 时间片用完
- ☒ B 等待某事件发生
- ☐ C 等待的某事件已发生
- ☐ D 被进程调度程序选中

提交

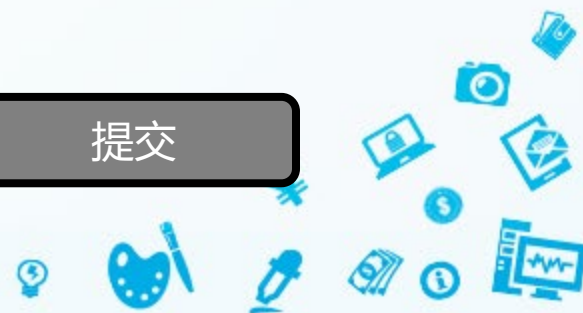


11. 下列的进程状态变化中，（ ）变化是不可能发生的

C

- ☐ A 运行→就绪
- ☐ B 运行→等待
- ☒ C 等待→运行
- ☐ D 等待→就绪

提交

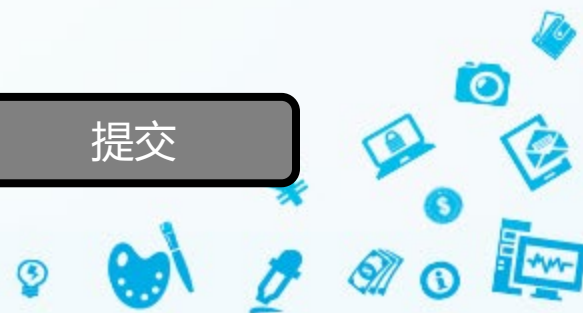


12. 一个运行的进程用完了分配给它的时间片后，它的状态变为（ ）

A

- ☒ A 就绪
- ☐ B 等待
- ☐ C 运行
- ☐ D 由用户自己确定

提交



处于创建状态的进程获取所需的资源并完成PCB的初始化工作后，由**创建状态**转入**就绪状态**

3. 创建状态和终止状态

□ 创建进程的过程：

- (1) 进程申请一个空白PCB
- (2) 向PCB中填写用于控制和管理进程的信息
- (3) 为该进程分配运行时所必须的资源
- (4) 把进程转入就绪状态并插入就绪管理队列中
- (5) 若进程所需的资源无法得到满足，如内存不足，此时创建工作尚未完成，进程不能被调度运行，此时进程所处的状态就是 **创建状态**



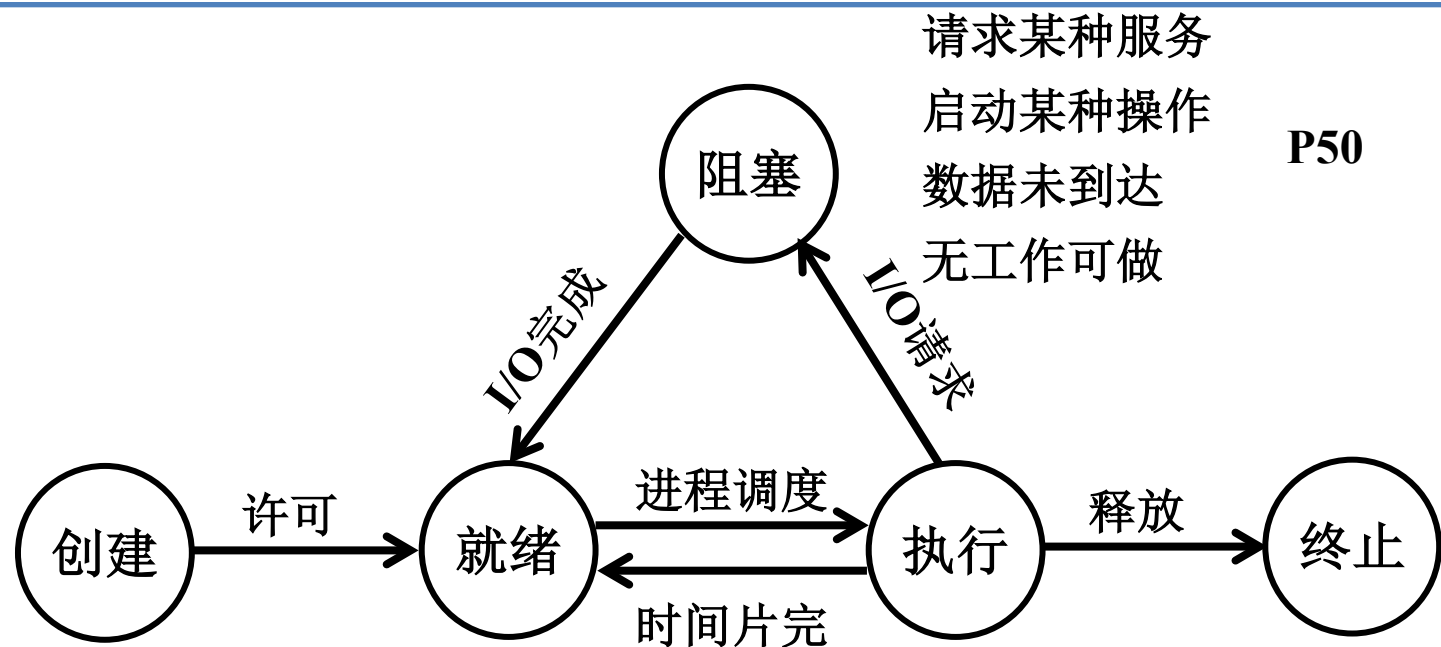
3. 创建状态和终止状态

❑ **终止状态**：当一个进程到达了自然结束点，或是出现了无法克服的错误，或是被操作系统所终结，或是被其他有终止权的进程所终结，它将进入**终止状态**

- 终止状态的进程不能再执行；
- 在操作系统中依然保留一个记录，其中保存状态码和一些计时统计数据，供其他进程收集；
- 一旦其他进程完成了对其信息的提取后，操作系统将删除该进程，将其PCB清零，并将该空白PCB返还系统。



3. 创建状态和终止状态



P49

用户登录
作业调度
提供服务
应用请求

P50

正常结束
异常结束
外界干预

P50



2.2.3 挂起操作和进程状态的转换

进程控制

- 1. 挂起操作的引入
- 2. 引入挂起原语操作后三个进程状态的转换
- 3. 引入挂起操作后五个进程状态的转换



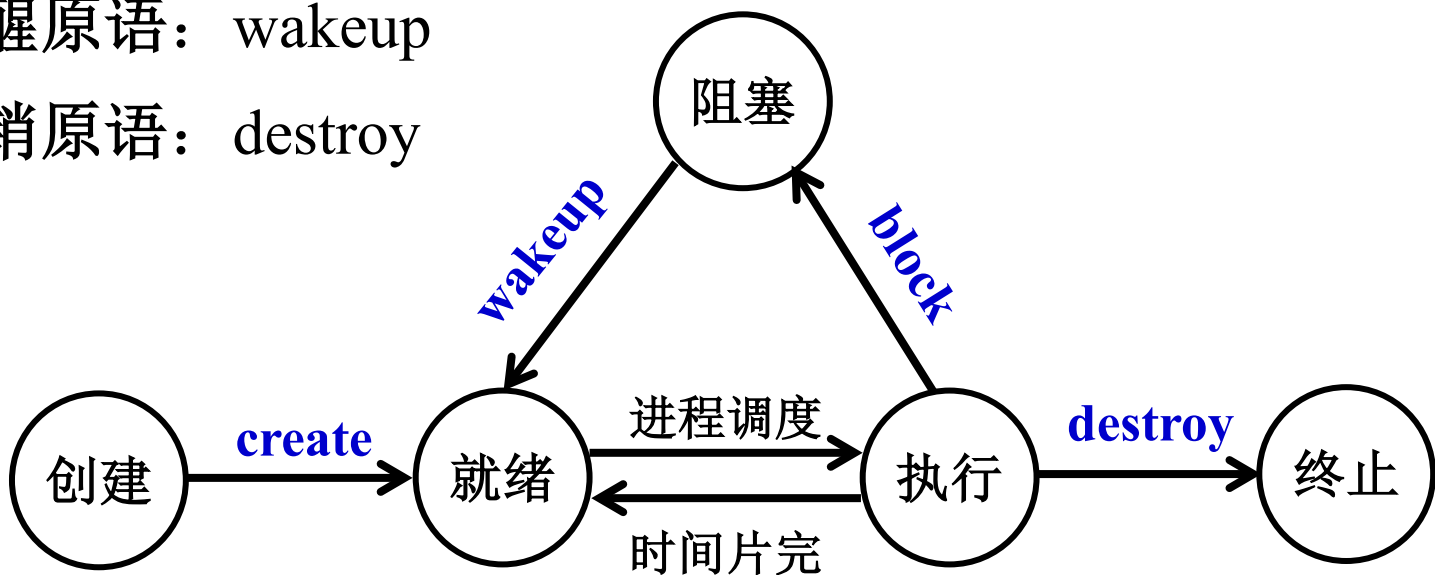
● 提前：进程控制 P47

- ❑ **进程控制**：即OS对进程实现有效的管理，包括创建新进程、终止已完成进程、挂起、阻塞和唤醒、进程切换等多种操作，是进程管理中最基本的功能。
- ❑ OS通过**原语（Primitive）**操作实现进程控制。
- ❑ **原语**：由若干条指令组成的程序段，用于完成特定功能。
 - 是一种**原子操作**（Action Operation），要么全做，要么不做，执行过程不会被中断；
 - 在系统态（或内核态、管态）下执行，常驻**内存**；
 - 是内核三大支撑功能之一。



提前：进程控制 P47

- ❑ 创建原语：create
- ❑ 阻塞原语：block
- ❑ 唤醒原语：wakeup
- ❑ 撤销原语：destroy



第二章 进程的描述与控制

第2讲





知识回顾

- 进程的定义
- 进程的结构
- 进程的特征
- 进程的三个基本状态
- 进程的五个基本状态
- OS内核
- 处理机的运行状态
- 原语操作



进程和程序的联系和区别

□ 联系：

- 程序是产生进程的基础；
- 程序的每次运行构成不同的进程；
- 进程是程序功能的体现；
- 通过多次执行，一个程序可对应多个进程；通过调用关系，一个进程可包括多个程序。



● 进程和程序的联系和区别

□ 区别：

- **进程**是**动态**的，**程序**是**静态**的：程序是有序代码的集合；进程是程序的执行，进程有核心态、用户态；
- **进程**是**暂时**的（有生命周期），**程序**是**永久**的：进程是一个状态变化的过程，程序可长久保存；
- **进程与程序的组成不同**：进程的组成包括程序、数据和**进程控制块**（即进程状态信息）



● 举例：科学家做蛋糕

有一个计算机科学家，想亲手给女儿做一个生日蛋糕。所以他开始准备做蛋糕所需物品：一本有关做蛋糕的食谱，一些原料（面粉、鸡蛋、糖、巧克力粉、奶油、柠檬、牛奶等）和工具（模具、搅棒器、烤箱等），一个记录制作过程的笔记本。

则：食谱 = （ 程序 ）；科学家 = （ CPU ）；

原料 = （ 数据 ）；工具 = （ I/O ）；

笔记本 = （ PCB ），

做蛋糕 = （ 进程 ），处于 （ 创建 ）状态



● 举例：科学家做蛋糕

半小时后，所需物品准备完毕。

此时，做蛋糕进程处于（就绪）状态；

然后，科学家开始边看边学边做，并将制作过程记录在笔记本上。

此时，做蛋糕进程处于（执行）状态；



● 举例：科学家做蛋糕

忽然，科学家的小儿子哭着跑进来，说手被蜜蜂蛰了。科学家只好把蛋糕先放到一边，并在笔记本上做了个标记，把状态信息记录了起来。然后他找出急救箱和急救手册，查到相关处理方法，按照上面的步骤一步步地执行。

则：做蛋糕进程处于（ **就绪** ）状态；

处理伤口进程处于（ **执行** ）状态。



● 举例：科学家做蛋糕

当伤口处理完之后，科学家回到厨房，找到之前记下的蛋糕制作笔记，接着之前的进度继续制作蛋糕。过了不久，科学家将制作好的蛋糕糊倒入模具，然后放入烤箱进行烘烤。

此时，做蛋糕进程处于（ **阻塞** ）状态。

35分钟后，烤箱烘烤结束。

此时，做蛋糕进程处于（ **就绪** ）状态。

科学家取出蛋糕，然后参照食谱涂抹奶油、书写生日祝词。生日蛋糕制作完成后，科学家整理好材料、工具等。

此时，做蛋糕进程处于（ **终止** ）状态。



2.2.3 挂起操作和进程状态的转换

进程控制

- 1. 挂起操作的引入
- 2. 引入挂起原语操作后三个进程状态的转换
- 3. 引入挂起操作后五个进程状态的转换



1. 挂起操作的引入

- 挂起状态：把暂时不能运行的进程调至外存等待，此时进程的状态称为就绪驻外存状态（或挂起状态）。P93
- 引入挂起操作的原因：
 - （1）终端用户的请求：当终端用户有疑问，希望暂停执行。
 - （2）父进程请求：希望考察、修改子进程，或协调各子进程间的活动。
 - （3）负荷调节的需要：实时系统中工作负荷较重时，系统可把一些不重要的进程挂起。
 - （4）操作系统的需要：操作系统有时希望挂起某些进程，以便检查运行中的资源使用情况或进行记账。



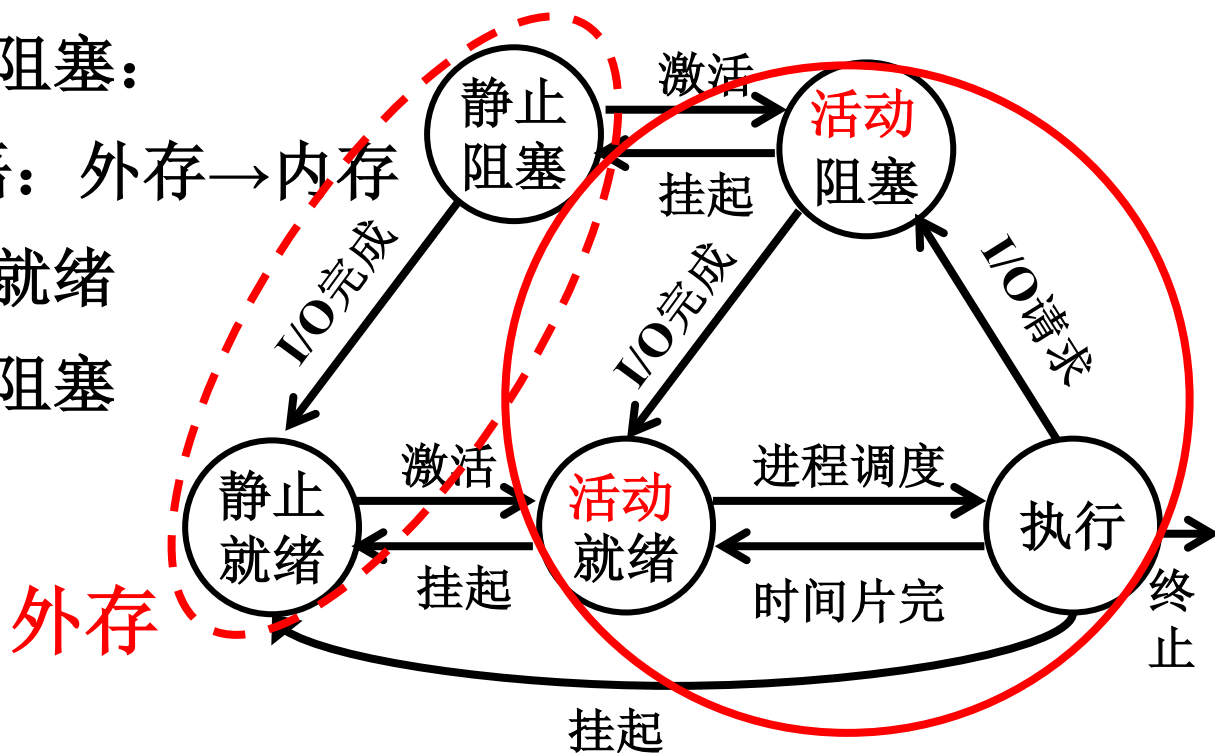
2. 引入挂起原语操作后三个进程状态的转换

□ 挂起 (Suspend) 原语：内存→外存

- 活动就绪→静止就绪：放外存，不调度
- 活动阻塞→静止阻塞：

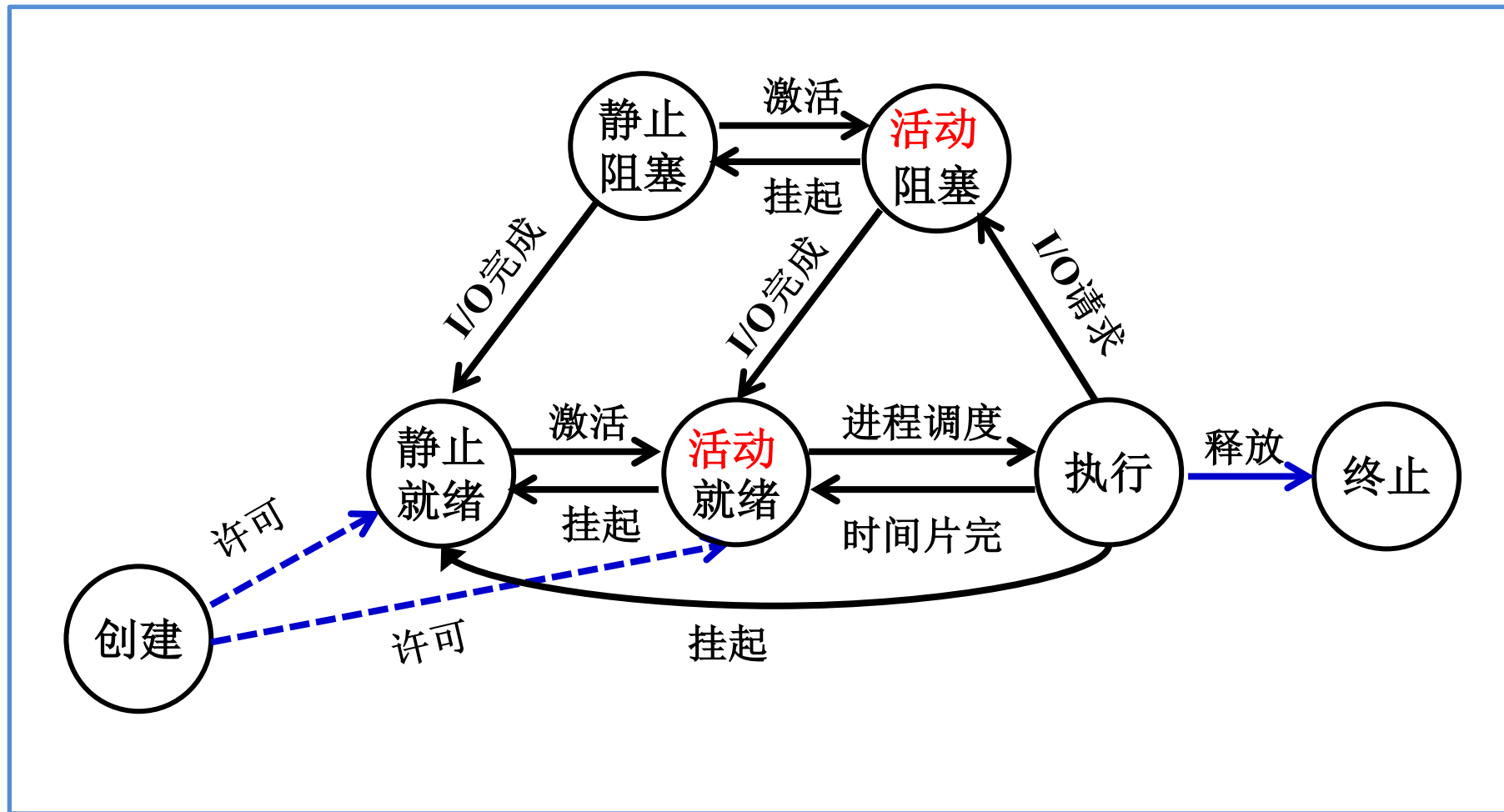
□ 激活 (Active) 原语：外存→内存

- 静止就绪→活动就绪
- 静止阻塞→活动阻塞





3. 引入挂起操作后五个进程状态的转换





思考



如果让你来设计一个OS，怎样实现其
进程管理机制？

程序 = 算法 + 数据结构



2.2.4 进程管理中的数据结构

- 1. OS中用于管理控制的数据结构
- 2. 进程控制块PCB的作用
- 3. 进程控制块中的信息
- 4. 进程控制块的组织方式



1. OS中用于管理控制的数据结构

- ❑ 资源（进程）信息表：计算机系统给每个资源和每个进程设置的一个数据结构，用于表征其实体。
- ❑ OS管理的数据结构分为：内存表、设备表、文件表和用于进程管理的**进程表**（又称为**进程控制块PCB**）。

2. 进程控制块PCB的作用

□ 进程控制块PCB：为了描述和控制进程的运行，系统为每个进程定义的一个数据结构。进程控制块是进程实体的一部分，是操作系统中最重要的记录型数据结构。

□ 作用：

使一个在多道程序环境下不能独立运行的程序（含数据），成为一个能独立运行的基本单位，一个能与其它进程并发执行的进程。

或者说，OS是根据PCB来对并发进程进行控制和管理。



3.进程控制块中的信息

- **PCB**中记录了操作系统所需的，用于**描述进程当前情况**以及**控制进程运行的全部信息**。具体包括下述四方面的信息：
- 1) 进程标识符（**PID: Process Identifier**）：**唯一标识进程**
外部标识符（名）；内部标识符（进程号）
 - 2) 处理机状态
 - 主要由处理机的各种寄存器中的内容组成的
 - 当处理机被中断时，这些信息都必须保存到PCB中，以便该进程重新执行时，能从断点继续执行



3.进程控制块中的信息

3) 进程调度信息

在PCB中还存放一些与**进程状态和进程调度**有关的信息。包括：

- **进程状态**：作为调度和对换时的依据。
- **进程优先级**：描述进程使用处理机的优先级别的一个**整数**，优先级高的进程优先获得处理机。
- **进程调度所需的其它信息**：与所采用的进程调度算法有关，如进程已等待**CPU**的时间总和等
- **事件**：进程由执行状态转变为阻塞状态所等待发生的事件，即**阻塞原因**。



3.进程控制块中的信息

4) 进程控制信息

- **程序和数据地址**：指程序和数据所在的内存或外存首地址；
- **进程同步和通信机制**：如信号量、消息队列指针等，它们可能全部或部分地存放在PCB中；
- **资源清单**：是一张列出了除CPU外的、进程所需的全部资源及已经分配到该进程的资源的清单；
- **链接指针**：它给出本进程（PCB）所在队列中下一个进程的PCB的首址。



2. 进程控制块PCB的作用

□ 作用：OS是根据**PCB**来对并发进程进行控制和管理

例如：进程调度；现场保护和恢复；进程同步和通信。

- (1) 独立运行基本单位的标志：**PCB**是进程存在于系统的**唯一标志**。
- (2) 能实现间断性运行方式：可以将**CPU**现场信息保存在**PCB**中
- (3) 提供进程管理所需要的信息：程序、数据和文件的地址信息等
- (4) 提供进程调度所需要的信息：进程的状态和优先级等
- (5) 实现与其他进程的同步与通信：信号量及通信队列指针等



4.进程控制块的组织方式

□ 常用的组织方式有三种：**线性方式**、链接方式和索引方式。

(1) 线性方式

将系统中的所有PCB都组织在一张线性表中，将该表的首指针放在内存一个专用的区域。

实现简单，开销小，但每次查找都需要扫描整张表，因此适合进程数目不多的系统。

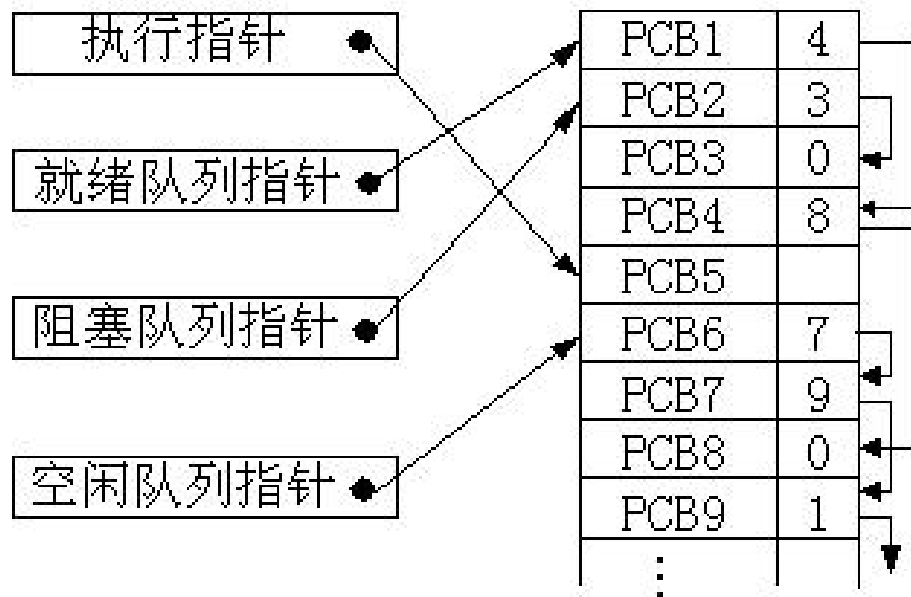


4.进程控制块的组织方式

□ 常用的组织方式有三种：线性方式、**链接方式**和索引方式。

(2) 链接方式

把具有同一状态的PCB，
用**PCB的链接字**链接成一个队列。形成就绪队列、
阻塞队列、空白队列等





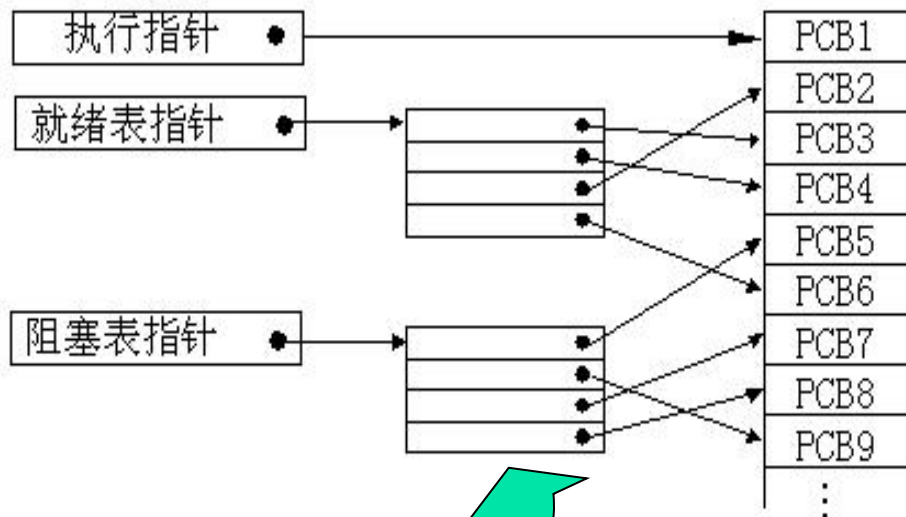
4.进程控制块的组织方式

□ 常用的组织方式有三种：线性方式、链接方式和索引方式。

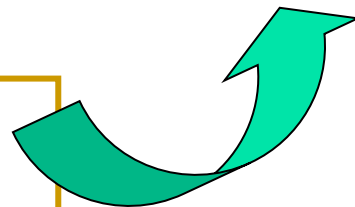
(3) 索引方式

系统根据所有进程的状态建立几张索引表。如，

- 就绪索引表
- 阻塞索引表等



- ▲索引表的首址记录在专用单元中；
- ▲每个索引表的表目中，记录具有相应状态的某个PCB在PCB表中的地址。





2.3 进程控制

- 2.3.1 操作系统内核
- 2.3.2 进程的创建
- 2.3.3 进程的终止
- 2.3.4 进程的阻塞与唤醒
- 2.3.5 进程的挂起与激活



2.3 进程控制

- 进程控制是进程管理中**最基本**的功能。
- 进程控制包括：**创建进程、终止进程、进程状态转换等**。
- 进程控制是由OS的**内核**完成的。
- 进程控制一般都是由OS内核中的**原语**来实现。



2.3.1 操作系统内核

- 1. 支撑功能
- 2. 资源管理功能



2.3.1 操作系统内核

- OS的内核：和**硬件紧密相关的模块**（如中断处理程序等）、**各种常用设备的驱动程序**，以及**运行频率较高的模块**（如时钟管理、进程调度和许多模块所用的一些基本操作），都安排在**紧靠硬件的软件层次**中，将它们常驻**内存**。
- 通常将**处理机的执行状态**分为**系统态**和**用户态**。
 - **系统态**：又称**管态**，也称**内核态**。具有较高特权，能执行一切指令，访问所有寄存器和存储区，传统OS都在系统态运行。
 - **用户态**：又称目态。具有较低特权的执行状态，仅能执行规定的指令，访问指定的寄存器和存储器。



2.3.1 操作系统内核

OS内核的两大方面功能：支撑和资源管理

1. 支撑功能

- (1) 中断处理： 内核**最基本的功能**，整个OS赖以活动的基础。
- (2) 时钟管理： 内核的一项**基本功能**
- (3) 原语操作： 由若干条指令组成的**程序段**，**原子操作**



2.3.1 操作系统内核

OS的两大方面功能：支撑和**资源管理**

2. 资源管理功能

- (1) 进程管理：通常放在内核中，以提高OS性能
- (2) 存储器管理：通常放在内核中，以保证存储器高速运行
- (3) 设备管理：大部分设置在内核中



2.3.2 进程的创建

- 1. 进程的层次结构
- 2. 进程图
- 3. 引起创建进程的事件
- 4. 进程的创建

【注意】 Windows中不存在进程层次结构的概念，所有的进程具有相同的地位

1. 进程的层次结构

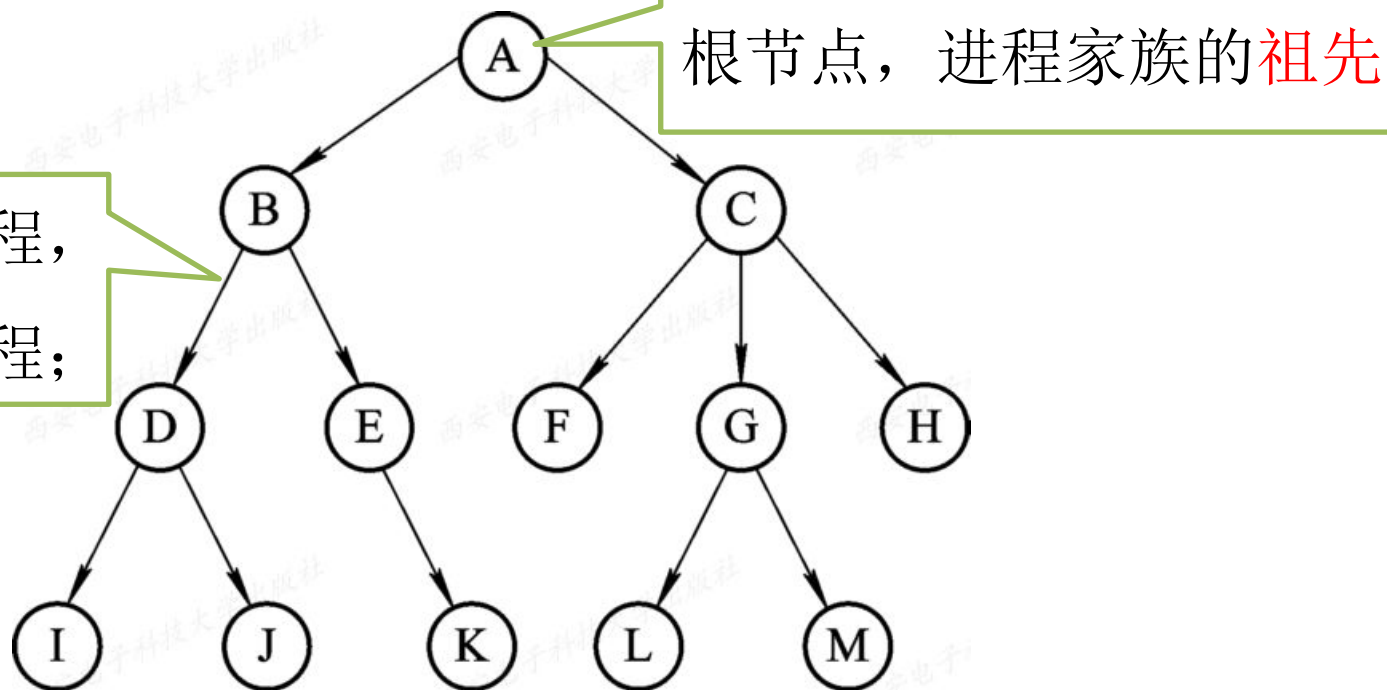
在OS中，允许一个进程创建另一个进程，通常把创建进程的进程称为**父进程**，而把被创建的进程称为**子进程**。子进程可继续创建更多的**孙进程**，由此便形成了一个进程的层次结构。

- 子进程可以继承父进程拥有的所有资源；子进程被撤销时，应将其从父进程哪里获得的资源归还给父进程。
- 撤销父进程时，必须同时撤销其所有的子进程。
- **PCB**设置了家族关系表项，以标明自己的父进程及所有的子进程。
- 父进程不能拒绝子进程的继承权。



2. 进程图

□ 进程图（Process Graph）：用于描述进程间关系的一棵有向树，用来描述一个进程的家族关系。





3. 引起创建进程的事件

为使程序之间能并发运行，应先为它们分别创建进程。导致一个进程去创建另一个进程的典型事件有四类：

(1) 用户登录

(2) 作业调度

(3) 提供服务

(4) 应用请求

当用户进程提出某种请求后，系统将专门创建一个进程来提供用户所需的服务。如，文件打印。

上述三种情况，都是由**系统内核**为它创建一个新进程。

基于应用进程的需求，由应用进程自己创建一个新进程，以便新进程以并发运行方式完成特定任务。



4. 进程的创建

□ 调用进程创建原语**Create**（），按下述步骤创建一个进程：

（1）申请空白PCB；

（2）为新进程分配资源。主要是内存空间；

（3）初始化PCB，包括：

初始化标识信息

初始化处理机状态信息：程序计数器，堆栈指针等

初始化处理机控制信息：进程状态——就绪或静止就绪、优先级等。

（4）将新进程插入就绪队列。



2.3.3 进程的终止

- 1. 引起进程终止的事件
- 2. 进程的终止过程



1. 引起进程终止的事件

(1) 正常结束

(2) 异常结束

异常结束事件有：

(3) 外界干预

- 操作员或操作系统干预（如用户结束、发生死锁）
- 父进程请求
- 父进程终止

- 越界错误
- 保护错：试图访问不允许访问的资源或文件，或者以不适当方式访问
- 非法指令
- 特权指令错：用户程序试图执行只允许OS执行的指令
- 运行超时
- 等待超时
- 算术运算错：如除以0
- I/O故障



2. 进程的终止过程

OS调用**终止原语**，按下述过程终止进程：

- （1）根据被终止进程的标识符，从**PCB**集合中检索出该进程的**PCB**，读出该进程状态。
- （2）若被终止进程正处于执行状态，应立即终止其执行，并置调度标志为真，用于指示该进程被终止后应重新进行调度。若该进程还有子孙进程，应将其所有子孙进程终止，以防止它们成为不可控进程。
- （3）将被终止进程的所有资源，或者归还给其父进程，或者归还给系统。
- （4）将被终止进程（其**PCB**）从所在队列中移出，等待其他进程来搜索信息。



2.3.4 进程的阻塞与唤醒

- 1. 引起进程阻塞与唤醒的事件
- 2. 进程阻塞过程
- 3. 进程唤醒过程



1. 引起进程阻塞与唤醒的事件

(1) 请求系统共享资源失败

进程请求共享资源时，系统无足够资源分配给进程。如，进程请求打印机

(2) 等待某种操作完成

当进程启动某种操作后，若必须在该操作完成后才能继续执行，则必须先进入阻塞状态，以等待操作完成。如，启动I/O设备

(3) 新数据尚未到达

对于相互合作的进程，若其中一个进程需要获得另一个进程提供的数据才能继续运行，则其所需数据尚未到达前，该进程只有阻塞。

(4) 等待新任务的到达

系统设置一些具有特定功能的系统进程，每当这种进程完成任务后，便把自己阻塞起来以等待新任务到来。如，系统中发送数据的进程



2. 进程阻塞过程

调用**阻塞原语block**把自己阻塞——**主动行为**。

□ 阻塞block（）执行过程：

- （1）立即停止执行；
- （2）把PCB中进程状态由“执行”改为“阻塞”；
- （3）将PCB插入具有相同事件的阻塞队列；
- （4）转进程调度程序，将处理机分配给某个就绪进程，并进行进程切换——保留被阻塞进程的处理机状态（在PCB中），再按新进程的PCB中处理机状态设置CPU的环境。



3. 进程唤醒过程

调用唤醒原语**wakeup()**，将等待事件的进程唤醒。

□ 唤醒原语执行过程：

- (1) 将被唤醒进程的**PCB**从阻塞队列移出；
- (2) 将其**PCB**中进程状态由“阻塞”改为“就绪”；
- (3) 将改**PCB**插入到就绪队列中。

block() 和 **wakeup()** 是成对的。



2.3.5 进程的挂起与激活

- 1. 进程的挂起
- 2. 进程激活过程



1. 进程的挂起

当出现引起进程挂起的事件时，系统将用挂起原语suspend()将指定进程或处于阻塞状态的进程挂起。

□ 挂起原语的执行过程：

(1) 检查被挂起进程的状态；

若处于活动就绪或执行状态，则将其转为静止就绪；
若处于活动阻塞，则将其转为静止阻塞。

(2) 把该进程的PCB复制到某指定内存区域；

为方便用户或父进程考查该进程的运行状态。

(3) 若该进程正在执行，则转进程调度程序重新调度。



2. 进程激活过程

当发生激活进程的事件时（如父进程或用户请求激活指定进程，而内存中已有足够空间时），系统利用激活原语`active()`将指定进程激活。

□ 激活原语的执行过程：

（1）将进程从外存调入内存；

（2）检查该进程现行状态；

若是静止就绪，则改为活动就绪；

若是静止阻塞，则改为活动阻塞。

（3）若采用抢占式调度策略，则检查被激活就绪进程的优先级，若其优先级比先行执行进程高，则将处理机分配给被激活进程。

1. 对进程的管理和控制使用（ ）。

B

- ☐ A 指令
- ☒ B 原语
- ☐ C 信号量
- ☐ D 信箱通信

提交

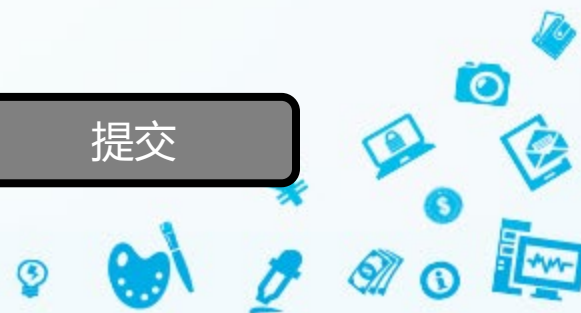


2. 进程控制就是对系统中的进程实施有效的管理，通过使用（ ）、进程撤销、进程阻塞、进程唤醒等进程控制原语实现。

C

- ☐ A 进程运行
- ☐ B 进程管理
- ☒ C 进程创建
- ☐ D 进程同步

提交



3. 操作系统通过 () 对进程进行管理。

B

- ☐ A 进程
- ☒ B 进程控制块
- ☐ C 进程启动程序
- ☐ D 进程控制区

提交



4. 将进程的 [填空1] 的链接字链接在一起就形成了进程队列。

PCB

作答

5. 通常，用户进程被建立后（ ）

B

- ☐ A 便一直存在于系统中，直到被操作人员撤销
- ☒ B 随着作业运行正常或不正常结束而撤销
- ☐ C 随着时间片轮转而撤销与建立
- ☐ D 随着进程的阻塞或唤醒而撤销与建立

提交



6. 设系统中有 $n(n > 2)$ 个进程，且当前不在执行进程调度程序，试考虑下述4种情况，不可能发生的情况是 **A**

A

没有运行进程，有2个就绪进程， $n-2$ 个进程处于阻塞状态

B

有1个运行进程，没有就绪进程， $n-1$ 个进程处于阻塞状态

C

有1个运行进程，有1个就绪进程， $n-2$ 个进程处于阻塞状态

D

有1个运行进程， $n-1$ 个就绪进程，没有进程处于阻塞状态

提交





2.4 进程同步

- 2.4.1 进程同步的基本概念
- 2.4.2 硬件同步机制
- 2.4.3 信号量机制
- 2.4.4 信号量的应用
- 2.4.5 管程机制



2.4 进程同步

多个进程必须以互斥的方式共享同一临界资源，即当一个进程正在使用临界资源且尚未使用完毕时，其他进程必须推迟对该资源的进一步操作，不能从中插进去使用这个临界资源，否则将会造成信息混乱和操作出错。

进程同步的主要任务：对多个相关进程在执行次序上进行协调，使并发执行的诸进程之间能按照一定的（或预期的）规则（或时序）有效的共享资源和相互合作，从而使程序的执行具有可再现性。





2.4.1 进程同步的基本概念

- 1. 两种形式的制约关系
- 2. 临界资源
- 3. 临界区
- 4. 同步机制应遵循的规则



1. 两种形式的制约关系

(1) 间接制约关系 进程**排他性**（或**互斥**）的访问临界资源，
进程互斥 间接制约关系源于**资源共享**。
如，共享打印机。

(2) 直接制约关系 源于**进程间的合作**。如，管道通信
进程同步

下列活动分别属于哪种制约关系？

- (1) 若干同学去图书馆借书； (2) 两队举行篮球比赛；
(3) 流水线生产的各道工序； (4) 商品生产和社会消费。



2. 临界资源

在一段时间内只允许一个进程访问的资源，即仅当一个进程访问完并释放该资源后，才允许另一个进程访问的资源，称为临界资源或独占资源。

如，打印机、磁带机、共享变量、队列、.....





2. 临界资源

□ 经典的生产者-消费者问题：

Dijkstra把广义同步问题抽象成一种“生产者与消费者问题”（Producer-consumer-relationship）的抽象模型。计算机系统中的一个问题都可归结为生产者与消费者问题。

如：

打印程序（进程）产生字符 → 供打印驱动程序（进程）使用；

编译器产生的汇编代码（汇编语言代码）→ 供汇编程序使用；

汇编程序产生的目标代码 → 供连接装入程序使用。



Dijkstra

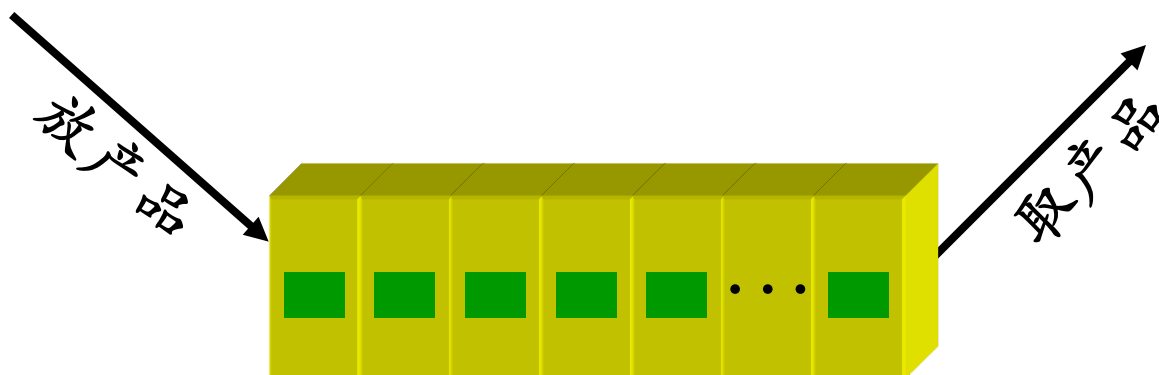


● 经典的生产者-消费者问题

- 问题描述：有一群生产者进程在生产产品，并将这些产品提供给消费者进程去消费。
- 为使生产者进程与消费者进程能并发执行，在两者之间设置了一个具有 n 个缓冲区的缓冲池；
- 生产者进程将所生产的产品放入一个缓冲区中；消费者进程可从缓冲区中取走产品去消费。
- 尽管所有的生产者进程和消费者进程都是以异步方式运行的，但它们之间必须**保持同步**，即不允许消费者进程到一个空缓冲区去取产品，也不允许生产者进程向一个已装满产品且尚未被取走的缓冲区投放产品。



经典的生产者-消费者问题



一次只能放一个产品；
一次只能取走一个产品；
缓冲区满则生产者不能放产品；
缓冲区空则消费者不能取产品；

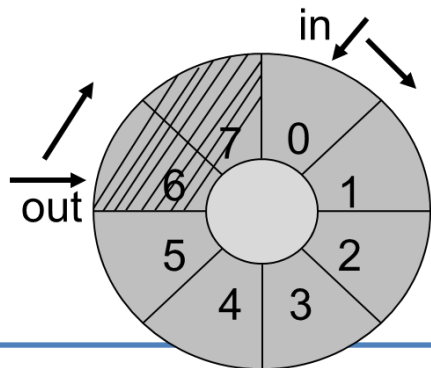


经典的生产者-消费者问题

□ 用数据模型来实现:

- 用数组buffer表示有n个（0,1,2,...n-1）缓冲区的缓冲池；
- 用输入指针in表示下一个可投放产品的缓冲区；
- 用输出指针out表示下一个可从中获取产品的缓冲区；
- 生产者进程投放一个产品，输入指针in加1；
- 消费者进程取走一个产品后，输出指针out加1；
- 由于缓冲池可以循环使用，所以需要循环的数据结构；

可用循环队列实现





生产者-消费者程序

- 数组buffer表示具有n个缓冲区的缓冲池；
- 每投入一个产品，缓冲池buffer中暂存产品的数字单元指针in加1；
- 每取出一个产品，缓冲池buffer中已取走产品的空闲单元指针out加1；
- 由于buffer组成的缓冲池是循环缓冲，因此输入指针in和输出指针out的加1操作应表示成

$in=(in+1)\%n$ 和 $out=(out+1)\%n$;

- 当 $(in+1) \% n = out$ 时表示缓冲池满，当 $out = in$ 时表示缓冲池空
- 缓冲池中产品数量为counter
- 生产者和消费者两个进程共享以下变量：

int in=0, out=0, counter=0;

item buffer[n];

- nextp是生产者中的局部变量，用于暂存每次刚刚生产出来的产品；
- nextc是消费者中的局部变量，用于暂存每次要消费的产品。



//共享数据

```
int in=0, out=0, counter=0;  
item buffer[n];
```



生产者-消费者程序

//生产者进程

```
producer( ){  
while(1){  
produce an item in nextp;  
...  
while (counter==n) ;  
buffer[in]=nextp;  
in=(in+1) % n;  
counter=counter+1;  
}}
```

//消费者进程

```
consumer( ){  
while(1){  
while (counter==0 );  
nextc=buffer[out];  
out=(out+1) % n;  
counter=counter-1;  
consume the item in nextc;  
...  
}}
```


解决问题的关键在于进程同步（合理的推进顺序），应将变量 **counter** 作为临界资源处理，让两个进程互斥地访问变量 **counter**。



生产者-消费者程序

两个合作的进程都要修改 **counter**

共享数据

```
int counter=0;
```

生产者进程

```
counter=counter+1;
```



```
register1 = counter ;  
register1 = register1 + 1 ;  
counter = register1;
```

消费者进程

```
counter=counter-1;
```



```
register2 = counter ;  
register2 = register2 - 1 ;  
counter = register2;
```

设 **counter** 当前值为5

两个进程执行一次后，正确结果应为5

若按下列顺序执行：

```
register1 = counter; (5)  
register1 = register1 + 1; (6)  
register2 = counter; (5)  
register2 = register2 - 1; (4)  
counter = register1; (6)  
counter = register2; (4)
```

counter 的值为4，出错。

其他情况？

第二章 进程的描述与控制

第3讲





知识回顾

- 进程控制块PCB
- 进程控制
- 进程同步

7. 两个进程合作完成一个任务。在并发执行中，一个进程要等待其合作伙伴发来消息，或者建立某个条件后再向前执行，这种制约性合作关系被称为进程的（ ）

- ☒ A 同步
- ☐ B 互斥
- ☐ C 调度
- ☐ D 执行

提交



8. 有两个用户进程A和B，在运行过程中都要使用系统中的一台打印机输出计算结果，则A、B两进程之间为（ ）制约关系

☐ A 直接

☒ B 间接

提交





//共享数据

```
int in=0, out=0, counter=0;  
item buffer[n];
```



生产者-消费者程序

//生产者进程

```
producer( ){  
while(1){  
produce an item in nextp;  
...  
while (counter==n) ;  
buffer[in]=nextp;  
in=(in+1) % n;  
counter=counter+1;  
}}
```

//消费者进程

```
consumer( ){  
while(1){  
while (counter==0 );  
nextc=buffer[out];  
out=(out+1) % n;  
counter=counter-1;  
consume the item in nextc;  
...  
}}
```

临界资源的互斥访问只需做到：访问前申请资源，可用时使用并加锁；用完释放资源并解锁。即进程互斥进入临界区。

3. 临界区（critical section）

□ 定义：进程中访问临界资源的那段代码称为临界区。

A: begin

Input data 1 form I/O 1 ;

Compute.....;

Print results 1 by printer ; 临界区A

End

B: begin

Input a data 2 form I/O 2 ;

Compute.....;

Print results 2 by printer ; 临界区B

End

进程互斥——不允许两个或两个以上进程同时进入相关临界区。

3. 临界区（critical section）

- ❑ 方法：若能保证诸进程互斥地进入自己的临界区，便可实现对临界资源的互斥访问。
- ❑ 实现思路：
 - 进入区（entry section）：位于临界区前，用于检查临界资源是否被占用的一段代码。每个进程在进入临界区之前，先利用进入区对欲访问的临界资源进行检查，看是否正被访问，若此刻该资源未被访问，便可进入临界区对该临界资源进行访问，并设置正被访问的标志（即加锁）；若此刻正被访问，则本进程不能进入临界区。
 - 退出区（exit section）：位于临界区后面，用于将临界区正被访问的标志恢复为未被访问标志（解锁）的一段代码。



3. 临界区 (critical section)

□ 访问临界区的循环进程:

```
While(TRUE)
{
```

进入区 (entry section)

临界区 (critical section)

退出区 (exit section)

剩余区 (remainder section)

```
}
```

用于对临界资源进行检查，即，尝试申请资源，**成功则加锁**

访问资源

将临界区正被访问的标志恢复为未被访问的标志，即，释放资源，**解锁**



4. 同步机制应遵循的规则

- 为实现各进程互斥地进入自己的临界区，设置专门的**同步机制**协调各进程间的运行。所有同步机制应遵循如下4条准则：
 - (1) **空闲让进** 当无进程处于临界区，应允许一个请求进入临界区的进程立即进入自己的临界区，以便有效地利用临界资源。
 - (2) **忙则等待** 当已有进程进入临界区，其他试图进入临界区的进程必须等待，以保证对临界资源的互斥访问。
 - (3) **有限等待** 对要访问临界资源的进程，应保证在有限的时间内能进入自己的临界区，以免陷入“**死锁**”状态——**不死等**。
 - (4) **让权等待** 当进程不能进入自己的临界区时，应立即释放处理机，以免进程陷入“**忙等**”。——不忙碌等待。

9. 下面关于临界区的叙述中，正确的是（ ）

- ☐ A 临界区可以允许规定数目的多个进程同时执行
- ☐ B 临界区只包含一个程序段
- ☒ C 临界区是必须互斥地执行的程序段
- ☐ D 临界区的执行不能被中断

提交



10. 若系统中有五个并发进程涉及某个相同的变量A，则变量A的相关临界区最少是由（ ）临界区构成。

- ☐ A 2个
- ☐ B 3个
- ☐ C 4个
- ☒ D 5个

提交





2.4.2 硬件同步机制（自学）

- 1. 关中断
- 2. 利用Test-and-Set指令实现互斥
- 3. 利用Swap指令实现进程互斥



2.4.3 信号量机制

- 1. 整型信号量
- 2. 记录型信号量
- 3. AND型信号量
- 4. 信号量集

进程同步的主要任务：对多个相关进程在**执行次序**上进行协调，使并发执行的**诸进程之间能按照一定的（或预期的）规则（或时序）**有效的**共享资源**和**相互合作**，从而使程序的执行具有可再现性。



进程同步

进程同步的主要任务：对多个相关进程在**执行次序**上进行协调，使并发执行的**诸进程之间能按照一定的（或预期的）规则（或时序）**有效的**共享资源**和**相互合作**，从而使程序的执行具有可再现性。

需要让进程“**走走停停**”来保证多进程**合作的合理有序**。

因此，要找到哪些地方要**停**，什么时候（**发信号**）再走？



阻塞



唤醒



如何实现上述操作？

根据共享资源的可用数量。

值为**2**，**0**，**-1**表示什么？  **信号量**

11. 假设某资源的总数量是6，该资源的信号量的当前值为2，说明（ ）。

- ☐ A 有2个进程等待该资源
- ☒ B 有2个可用资源
- ☐ C 有4个进程等待该资源
- ☐ D 有4个可用资源

提交





2.4.3 信号量机制

信号量（**Semaphores**）机制是一种卓有成效的**进程同步工具**。1965年由荷兰学者Dijkstra提出的一种**特殊整型变量**，表示**可用资源数目**，信号用来阻塞和唤醒，量用来记录。

- 信号量的产生源自于交通信号灯。
- 信号量是**被保护的数据结构**。除信号量的赋初值外，信号量的值仅能由两个同步原语改变。
- Dijkstra将这两个同步原语命名为“**P操作（又称wait操作）**”和“**V操作（又称signal操作）**”。

（P、V来源于荷兰文“发信号”和“等待”的首字母）



2.4.3 信号量机制

□ 信号量机制的发展：

1. 整型信号量 “忙等”，未遵循“让权等待”准则。
2. 记录型信号量 ——后边重点介绍。
3. AND型信号量
4. 信号量集

记录型信号量的扩展。

整型信号量机制中的wait操作，只要信号量 $S \leq 0$ ，就不断地测试。因此，该机制未遵循“**让权等待**”准则，使进程处于“**忙等**”。

1. 整型信号量

❑ 思想：Dijkstra把**整型信号量**定义为一个用于表示**资源数目**的**整型量S**，它与一般整型量不同，除初始化外，仅能通过两个标准的原子操作wait(S)和signal(S)来访问。

wait(S)

```
{  
while ( S<=0);  
S=S-1;  
}
```

P(S)

相当于进入区

signal(S)

```
{  
S=S+1;  
}
```

V(S)

相当于退出区

wait(s)和signal(s)是两个原子操作，执行时不能中断。即，当一个进程使用上述一个原语修改某信号量时，其他进程不能同时对该信号量进行修改。



1. 整型信号量

□ 整型信号量的应用

```
int s =1;  
Printer( )  
{  
    wait(s); //进入区，申请资源  
    print the document on the paper; //临界区，访问资源  
    signal(s); //退出区，释放资源  
}
```

检查和上锁一气呵成，
避免并发、异步导致的
问题



2. 记录型信号量

- 思想：需要一个用于代表资源数目的整型变量value、一个在该资源上阻塞的队列（链表）指针L。故信号量应采用记录型（C语言中为结构型）的数据结构：

/*记录型信号量的结构定义*/

```
typedef struct{  
    int value; //可用资源数目  
    /*阻塞队列的指针*/  
    struct process_control_block *list  
} semaphore ;
```

除初始化外，信号量只能通过wait (S) 和 signal (S) 原语来访问。它们以前被称为P、V操作。

S->value的初值表示系统中某类资源的总数目，即资源信号量。

2. 记录型信号量

□ wait和signal操作可用C/C++语言描述如下：

```
/*进程申请使用资源*/  
wait (semaphore *S){  
    S->value -- ;  
    if (S->value < 0 )  
        block (S->list) ;  
    /*自我阻塞，让权等待 */  
}
```

P(S)

相当于进入区

```
/*进程释放资源*/  
signal (semaphore *S){  
    S->value ++ ;  
    if (S->value <= 0 )  
        wakeup (S->list) ;  
    /*唤醒第一个等待的进程 */  
}
```

V(S)

相当于退出区

S.value的初值表示系统中某类资源的总数目，即资源信号量。

2. 记录型信号量

□ wait和signal操作可用C/C++语言描述如下：

```
/*进程申请使用资源*/  
wait (semaphore S){  
    S.value -- ;  
    if (S.value <0 )  
        block (S.list) ;  
    /*自我阻塞，让权等待 */  
}
```

P(S)

相当于进入区

```
/*进程释放资源*/  
signal (semaphore S){  
    S.value ++ ;  
    if (S.value <=0 )  
        wakeup (S.list) ;  
    /*唤醒第一个等待的进程 */  
}
```

V(S)

相当于退出区



2. 记录型信号量

□ wait和signal操作可用C/C++语言描述如下：

```
/*进程申请使用资源*/  
wait (semaphore *S){  
    S->value -- ;  
    if (S->value < 0 )  
        block (S->list) ;  
    /*自我阻塞，让权等待 */  
}
```

P(S)

相当于进入区

- 对信号量S的每次wait操作，意味着进程请求一个该类资源，使系统中可供分配的该类资源数减少一个，因此描述为S->value--;
- S->value<0时，表示该类资源已经分配完毕。因此进程应调用block原语进行自我阻塞，放弃处理机，并插入到阻塞队列链表S->list中。

S->value的绝对值表示在该信号量链表中已阻塞进程的数目。

2. 记录型信号量

□ wait和signal操作可用C/C++语言描述如下：

- 对信号量S的每次signal操作，意味着进程释放一个资源，使系统中可供分配的该类资源数增加一个，因此S->value++操作表示资源数目加1。
- 若加1后仍是S->value≤0，则表示该信号量阻塞链表中仍有等待该资源的进程被阻塞，故还应调用wakeup原语，将S->list链表中的第一个等待进程唤醒。

```
/*进程释放资源*/  
signal (semaphore *S){  
    S->value ++ ;  
    if (S->value <=0 )  
        wakeup (S->list) ;  
/*唤醒第一个等待的进程 */  
}
```

V(S)

相当于退出区

如果 $S \rightarrow \text{value}$ 的初值为1，表示只允许一个进程访问临界资源，此时的信号量转化为互斥信号了，用于进程互斥。



2. 记录型信号量

若S的初值为1，情况如何？

□ 记录型信号量的应用：

```
struct semaphore S;
```

```
S.value=3;
```

```
Printer()
```

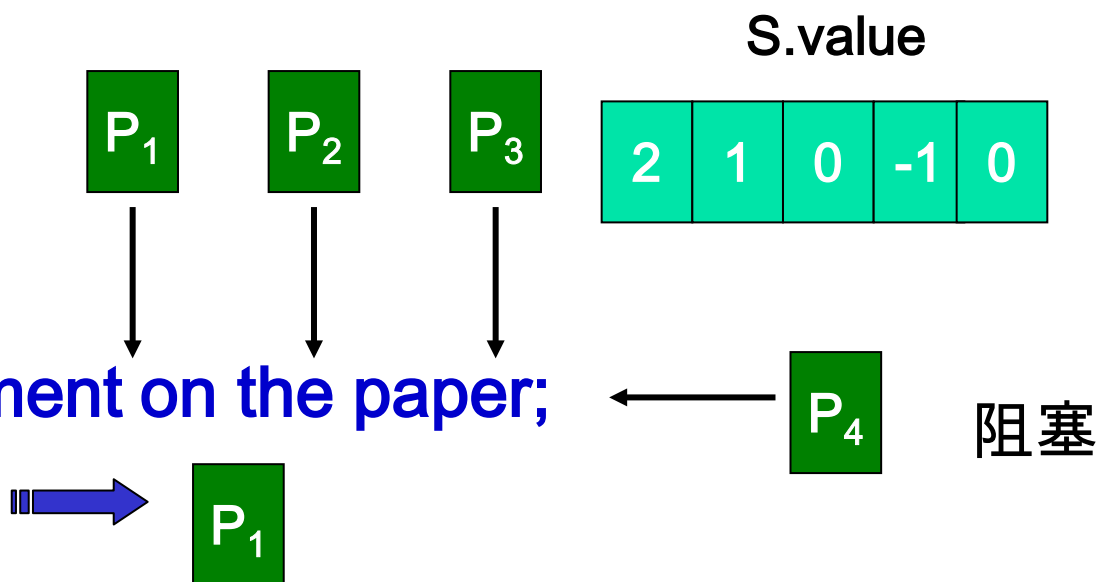
```
{
```

```
wait(S);
```

```
print the document on the paper;
```

```
signal(S);
```

```
}
```





2. 记录型信号量

□wait和signal操作的物理意义:

- Wait操作是申请资源的操作，是减1的操作。
- Signal操作是释放资源的操作，是加1的操作



2. 记录型信号量

□wait和signal操作的物理意义：

- 一般每一类资源对应一个信号量；
- 整型变量（信号量的值）代表了可用资源的数目；
- 等待队列里链接的是等待使用这种资源的进程；
- 整型变量的值如果大于零，代表有多个资源可用；
- 整型变量的值如果等于零，代表无可用资源；
- 整型变量的值如果小于零，代表有进程在等待资源，此时，整型变量的绝对值是等待（阻塞）队列中进程的数目。



3. AND型信号量

前面所述的进程互斥问题针对的是多个并发进程仅共享一种临界资源的情况。在有些应用场合，是一个进程往往需要获得两种或更多的共享资源后方能执行其任务。

例：假定现有两个进程A和B，它们都要求访问共享数据D和E，当然，共享数据都应作为临界资源。可为这两个数据分别设置两个信号量Dmutex和Emutex来实现互斥，它们的初值都是1。

```
A () {  
    D;  
    E;  
}
```

```
B () {  
    E;  
    D;  
}
```



3. AND型信号量

□ 例

```
A ( ){  
    wait(Dmutex);  
    wait(Emutex);  
    D;  
    E;  
    signal(Dmutex);  
    signal(Emutex);  
}
```

```
B ( ){  
    wait(Emutex);  
    wait(Dmutex);  
    E;  
    D;  
    signal(Emutex);  
    signal(Dmutex);  
}
```



3. AND型信号量

□ 例

Process A:
wait(Dmutex);
wait(Emutex);

Process B:
wait(Emutex);
wait(Dmutex);

若A和B按以下次序执行wait操作：

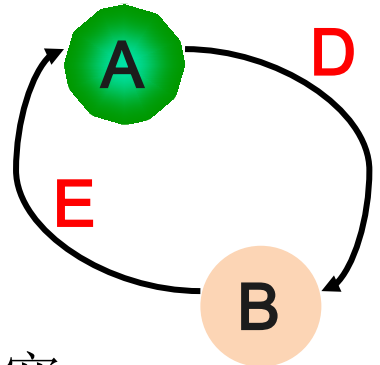
Process A: wait(Dmutex); 此时 Dmutex=0;

Process B: wait(Emutex); 此时 Emutex=0;

Process A: wait(Emutex); 此时Emutex=-1; A阻塞;

Process B: wait(Dmutex); 此时Dmutex=-1; B阻塞;

陷入僵局





3. AND型信号量

□思想：将进程在整个运行过程中需要的所有资源，一次性全部地分配给进程，待进程使用完成后再一起释放。只要尚有一个资源未能分配给进程，其他所有可能为之分配的资源，也不分配给它。即，**对若干个临界资源的分配，采取原子操作方式：要么全部分配给进程，要么一个也不分配**。为此，在wait操作中增加了一个“AND”条件，故称为AND同步，或称为同时wait操作，即Swait（simultaneous wait）



And型信号量的P操作

```
Swait( $S_1, S_2, \dots, S_n$ )
```

```
{
```

```
  while(TRUE)
```

```
  {
```

```
    if  $S_1 \geq 1 \ \& \dots \& \ S_n \geq 1$  {
```

```
      for( $i=1; i \leq n; i++$ )
```

```
         $S_i--$ ;
```

```
      break;
```

```
    }
```

```
  else{
```

place the process in the waiting queue associated with the first S_i found with $S_i < 1$, and set the program counter of this process to the beginning of Swait operation

```
  }
```

```
}
```

```
}
```

P操作

将相关的所有进程移入第一个 $S_i < 1$ 的等待队列中，并将该进程的PC指针指向Swait操作开始处



And型信号量的V操作

Ssignal($S_1, S_2, \dots, S_n$)

{

while(TRUE)

{

for($i=1; i \leq n; i++$){

S_i++ ;

remove all the process waiting in the queue associated with S_i into the ready queue.

}

}

}

V操作

把阻塞队列里所有和 S_i 相关的进程送进就绪队列



4. 信号量集

记录型信号量机制中，`wait(S)`或`signal(S)`操作**仅能对信号量施以加1或减1操作**，意味着每次只能对某类临界资源进行一个单位的申请或释放。当一次需要N个单位时，便要进行N次`wait(S)`操作，这显然是低效的，甚至会增加死锁的概率。

此外，有些情况下，为确保系统的安全性，当所申请的资源数量低于某一下限值时，还必须进行管制，不予以分配。

因此，当进程申请某类临界资源时，在每次分配之前，都必须测试资源的数量，判断是否大于可分配的下限值，决定是否予以分配。



4. 信号量集

- 思想：对AND信号量机制加以改进，对进程所申请的所有资源以及每类资源不同的资源需求量，在一次P、V原语操作中完成申请或释放。
- 进程对信号量 S_i 的测试值不再是1，而是该资源的分配下限值 t_i ，即要求 $S_i \geq t_i$ ，否则不予分配。
 - 一旦允许分配，进程对该资源的需求值为 d_i ，即表示资源进行 $S_i = S_i - d_i$ 操作，而不是简单的 $S_i = S_i - 1$ 操作。
 - 由此形成一般化的“信号量集”机制。

以下的程序中，**S**为信号量，**t**为下限值，**d**为需求值。

● 信号量集的P操作

Swait($S_1, t_1, d_1, \dots, S_n, t_n, d_n$)

if $S_1 \geq t_1$ and ... and $S_n \geq t_n$ then

for $i=1$ to n do

$S_i = S_i - d_i;$

endfor

else

Place the executing process in the waiting queue of the first S_i with $S_i < t_i$ and set its program counter to the beginning of the Swait Operation.

endif

将正在执行的进程移入第一个 $S_i < t_i$ 的等待队列中，并将该进程的PC指向Swait操作开始处

P操作



信号量集的V操作

Ssignal($S_1, d_1, \dots, S_n, d_n$)

for $i=1$ to n do

$S_i = S_i + d_i$;

Remove all the process waiting in the queue
associated with S_i into the ready queue

endfor;

V操作

以下的程序中，**S**为信号量，**t**为下限值，**d**为需求值。

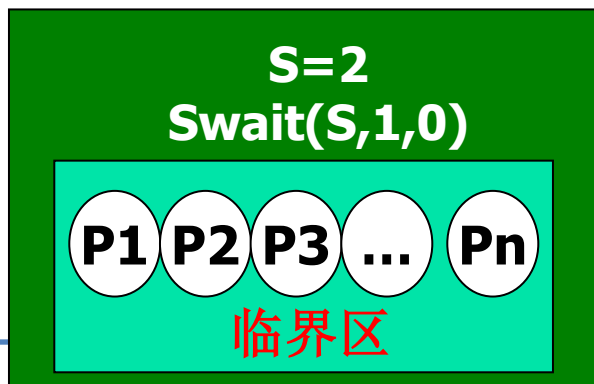
4. 信号量集

□ 信号量集的几种特殊情况：

(1) $\text{Swait}(S, d, d)$ ：信号量集中只有一个信号量**S**，但允许它每次申请**d**个资源，当现有资源小于**d**时，不予分配；

(2) $\text{Swait}(S, 1, 1)$ ：等同于一般的记录型信号量（ $S > 1$ ）或互斥信号量（ $S = 1$ ）；

(3) $\text{Swait}(S, 1, 0)$ ：当 $S \geq 1$ 时，允许多个进程进入临界区；当**S**为**0**，阻止所有进程进入临界区。其功能类似于可控开关。



12. () 是一种只能进行P操作和V操作的特殊变量。

- ☐ A 调度
- ☐ B 进程
- ☐ C 同步
- ☒ D 信号量

提交



13. 若P、V操作的信号量S初值为2，当前值为-1，则表示有（ ）等待进程。

- ☐ A 0个
- ☒ B 1个
- ☐ C 2个
- ☐ D 3个

提交



14. 对于两个并发进程，设互斥信号量为mutex，若 $\text{mutex}=0$ ，则（ ）

- ☐ A 表示没有进程进入临界区
- ☒ B 表示有一个进程进入临界区
- ☐ C 表示有一个进程进入临界区，另一个进程等待进入
- ☐ D 表示有两个进程进入临界区

提交



15. 操作系统中，对信号量S的P原语操作定义中，使进程进入相应等待队列等待的条件是（ ）

- ☐ A $S > 0$
- ☐ B $S = 0$
- ☒ C $S < 0$
- ☐ D $S \leq 0$

提交



一次仅允许一个进程访问的资源

复习

可用资源的数目

1. 信号量的物理意义是当信号量值大于零时表示__①__；当信号量值小于零时，其绝对值为__②__。
2. 临界资源的概念是__①__，而临界区是指__②__。
3. 若一个进程已进入临界区，其他欲进入临界区的进程必须等待。
4. 用P、V操作管理临界区时，任何一个进程在进入临界区之前应调用__P__操作，退出临界区时应调用__V__操作。
5. 有m个进程共享同一临界资源，若使用信号量机制实现对临界资源的互斥访问，则信号量值的变化范围是__[-(m-1), 1]__。
6. 操作系统中，对信号量S的P原语操作定义中，使进程进入相应等待队列等待的条件是__S<0__。





2.4.4 信号量的应用

□ 1. 利用信号量实现进程互斥

互斥问题

□ 2. 利用信号量实现前趋关系

同步问题

□ 3. 利用信号量控制进程资源的数量

资源管控问题



● 补充：信号量的类型及意义

- (1) **互斥信号量**：又称二值信号量。用于**申请或释放资源的使用权**，常初始化为**1**。
- (2) **资源信号量**：用于**申请或归还资源**，可初始化为**大于1的整数**，表示**系统中某资源的可用个数**。

说明：信号量的值小于0，其绝对值为阻塞队列中进程个数。

- **Wait操作（P操作）**：用于**申请资源（或使用权）**，若资源不够就阻塞自己。
- **Signal操作（V操作）**：用于**释放资源（或归还资源使用权）**，若有进程在等待该资源，则唤醒一个阻塞进程。



1. 利用信号量实现进程互斥

互斥问题

□ 方法要点:

- 为临界资源设置一个互斥信号量mutex，其初值为1；
- 各进程访问该资源的临界区置于 **wait(mutex)** 和 **signal(mutex)** 之间即可。

wait(mutex): 进入区
signal(mutex): 退出区

利用信号量实现 m 个进程互斥时，互斥信号量的取值范围为：
[-(m-1), 1]

1. 利用信号量实现进程互斥

□ 利用互斥信号量mutex实现两个进程互斥的描述：

- mutex的初值为1，取值范围为（-1， 0， 1）；
- mutex=1：表示两个进程都未进入临界区；
- mutex=0：表示已有一个进程进入临界区运行；
- mutex=-1：表示有一个进程正在临界区运行，另一个进程因等待而阻塞在信号量队列中，需要被当前已在临界区运行的进程在退出时唤醒。

利用信号量实现进程互斥时，wait(mutex)和signal(mutex)必须在**临界区的前后成对出现**。

1. 利用信号量实现进程互斥

□ 代码描述：

```
semaphore mutex=1;  
Pa( ){  
    while(1){  
        wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区:  
    }  
}
```

```
Pb( ){  
    while(1){  
        wait(mutex);  
        临界区;  
        signal(mutex);  
        剩余区:  
    }  
}
```

- 缺少wait(mutex) 不能保证对临界资源的互斥访问；
- 缺少signal(mutex)导致临界资源永不被释放，等待进程永不被唤醒。



1. 利用信号量实现进程互斥

□ 利用信号量实现进程互斥方法总结：

- (1) 分析问题，确定进程数、进程活动以及临界区；
- (2) 设置互斥信号量，初值为1；
- (3) 临界区前对信号量进行wait操作（P操作）；
- (4) 临界区后对信号量进行signal操作（V操作）。



1. 利用信号量实现进程互斥

【举例】某交通路口设置一个自动计数系统，系统由“观察者”进程和“报告者”进程组成。观察者进程能识别卡车，并对通过的卡车计数；报告者进程定时将观察者的计数值打印输出，每次打印后把计数值清“0”。两个进程的并发执行可完成对每小时中卡车流量的统计。

process observer:

计数值加1;

process reporter:

打印计数值;

计数值清零;

第1步：搞清楚谁是临界资源

计数值count

第2步：搞清楚哪些是临界区

访问count的语句，包括累加、

1. 利用信号量实现进

打印和清零

【举例】某交通路口设置一个自动计数系统，系统由“观察者”进程和“报告者”进程组成。观察者进程能识别卡车，并对通过的卡车计数；报告者进程定时将观察者的计数值打印输出，每次打印后把计数值清“0”。两个进程的并发执行可完成对每小时中卡车流量的统计。

```
struct semaphore S ;  
int count = 0 ;  
S.value = 1 ;
```

临界区

```
process observer  
{ while (condition){  
    observe a lorry;  
    wait (S) ;  
    count = count + 1;  
    signal (S) ;  
}}
```

```
process reporter  
{while (condition){  
    wait (S) ;  
    print (count) ;  
    count = 0;  
    signal (S) ;  
}}
```

临界区

16. 用P、V操作管理临界区时，信号量的初值应定义为（ ）

- ☐ A -1
- ☐ B 0
- ☒ C 1
- ☐ D 任意值

提交



17. 有 m 个进程共享同一临界资源，若使用信号量机制实现对临界资源的互斥访问，则信号量值的变化范围是
()

- ☒ A $[-(m-1), 1]$
- ☐ B $[-m, 1]$
- ☐ C $[-(m-1), 1]$
- ☐ D $[-m, 1)$

提交



思考：进程同步是什么？假设有并发执行的进程P1和P2， P1只有语句A， P2只有语句B。

2. 利用信号量实现前趋关系 同步问题

□ 希望先执行A，再执行B。为实现这种前趋关系，需要进行如下操作：

- 进程P1和P2共享一个公用**信号量S**，初值为0；
- **signal(S)**操作放在语句A后，语句B前插入**wait(S)**操作，即：

```
P1( ){  
    A ;  
    signal(S);  
}
```

```
P2( ){  
    wait(S);  
    B ;  
}
```

```
P1( ){  
    A ;  
}
```

```
P2( ){  
    B ;  
}
```

由于S被初始化为0，若P2先执行必定阻塞，只有在进程P1执行完A和**signal(S)**操作后使S增为1时，P2进程方能成功执行语句B。

假设有并发执行的进程P1和P2， P1只有语句A， P2只有语句B。

2. 利用信号量实现前趋关系 同步问题

□ 一次同步执行：执行顺序： $A \xrightarrow{S1} B$

一次性

```
struct semaphore S1;  
S1.value=0;  
P1( ){  
    A ;  
    Signal(S1);  
}
```

先运行的进程

```
P2( ){  
    Wait(S1);  
    B ;  
}
```

后运行的进程

思考：为什么是一次性的？如何实现循环同步？



2. 利用信号量实现前趋关系 同步问题

□ 利用信号量实现前趋关系（进程同步）方法总结：

- （1）分析问题，找到需要实现“**一前**一**后**”前趋关系（进程同步）的位置；
- （2）在**每个位置**上设置同步信号量，**初始值为0**；
- （3）在“**前操作**”之后执行**signal操作**（V操作）；
- （4）在“**后操作**”之前执行**wait操作**（P操作）。



2. 利用信号量实现前趋关系

同步问题

□ 思考：假设有并发执行的进程P1和P2，如图所示。要求按照下列顺序：A B D E F C

s1、s2的初始值为0



P1() {

A;

B;

signal(s1);

wait(s2);

C;

}

P2() {

wait(s1);

D;

E;

F;

signal(s2);

}

程序内部顺序性

程序外部异步性
信号量s1进行同步

程序内部顺序性

程序外部异步性
信号量s2进行同步

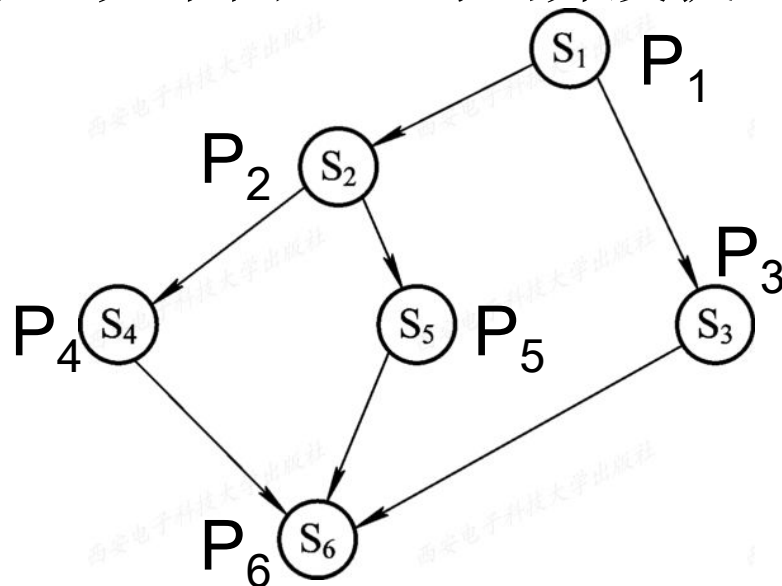
程序内部顺序性



2. 利用信号量实现前趋关系

□ 利用信号量按照语句间的前趋关系（见下图），写出并发执行的程序。

- 进程P1中有语句S1;
- 进程P2中有语句S2;
-
- 语句S1执行后才能执行语句S2和语句S3;
- 语句S2执行后才能执行语句S4和S5;
- 语句S3、S4和S5执行后，才能执行语句S6。



P1执行完应通知P2、P3;
P2得到通知后才开始执行;
P2执行完应通知P4、P5;
.....



2. 利用信号量实现前趋关系

□ 对应于每一对前趋关系，应分别设置信号量。

$S_1 \rightarrow S_2$: a

$S_1 \rightarrow S_3$: b

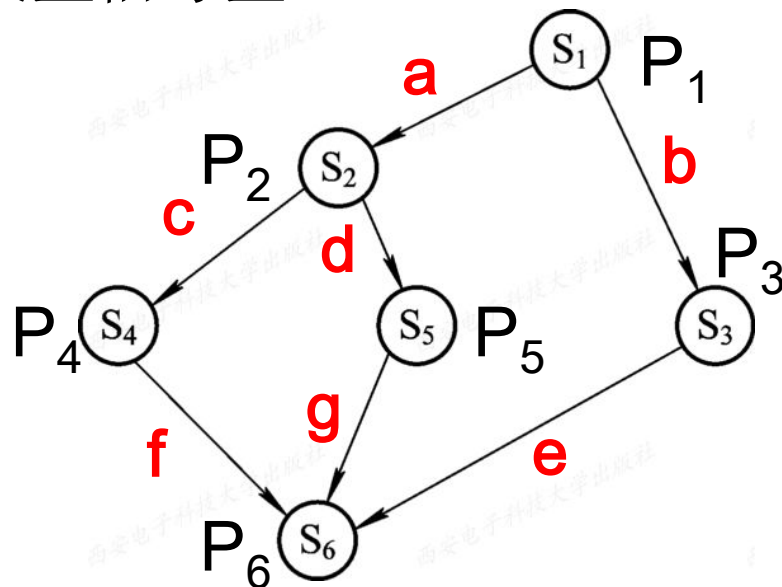
$S_2 \rightarrow S_4$: c

$S_2 \rightarrow S_5$: d

$S_3 \rightarrow S_6$: e

$S_4 \rightarrow S_6$: f

$S_5 \rightarrow S_6$: g



P1执行完应通知P2、P3;
P2得到通知后才开始执行;
P2执行完应通知P4、P5;
.....



2. 利用信号量实现前趋关系

```
struct semaphore a, b, c, d, e, f, g ;
    a.value = b.value
= c.value = d.value
= e.value = f.value
= g.value = 0;
```

初始化

begin /*begin表示并发执行开始*/

P1: { S1; signal(a); signal(b); }

P2: { wait(a); S2; signal(c); signal(d); }

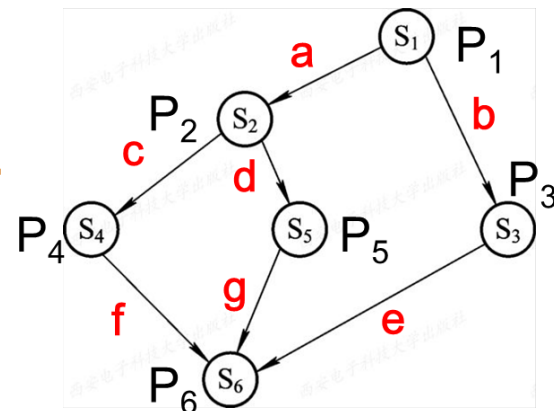
P3: { wait(b); S3; signal(e); }

P4: { wait(c); S4; signal(f); }

P5: { wait(d); S5; signal(g); }

P6: { wait(e); wait(f); wait(g); S6; }

end /*用end表示并发执行结束 */



$S_1 \rightarrow S_2$: a

$S_1 \rightarrow S_3$: b

$S_2 \rightarrow S_4$: c

$S_2 \rightarrow S_5$: d

$S_3 \rightarrow S_6$: e

$S_4 \rightarrow S_6$: f

$S_5 \rightarrow S_6$: g

假设有并发执行的进程P1和P2， P1只有语句A， P2只有语句B。

2. 利用信号量实现前趋关系 同步问题

□ 一次同步执行：执行顺序： $A \xrightarrow{S1} B$

一次性

```
struct semaphore S1;  
S1.value=0;  
P1( ){  
    A ;  
    Signal(S1);  
}
```

先运行的进程

```
P2( ){  
    Wait(S1);  
    B ;  
}
```

后运行的进程

思考：为什么是一次性的？如何实现循环同步？

假设有并发执行的进程P1和P2， P1只有语句A， P2只有语句B。

2+. 利用信号量实现循环进程同步

□ 循环同步执行： 执行顺序： $A \xrightarrow{S1} B \xleftarrow{S2}$

按照一定顺序
循环运行

```
struct semaphore S1, S2;
```

```
S1.value=0;
```

```
S2.value=1;
```

```
P1( ){
```

```
wait(S2);
```

```
A;
```

```
Signal(S1);
```

```
}
```

```
P2( ){
```

```
wait(S1);
```

```
B;
```

```
signal(S2);
```

```
}
```




3. 利用信号量控制使用资源进程的数量

- 系统中有5台打印机可以使用，请使用进程同步机制使得最多可以有5个进程可以同时使用打印机，多于5个进程使用时，要对新申请进程进行阻塞

程序描述如下：

```
struct semaphore S;  
S.value=5 ;  
Printer( ){  
    wait(S);  
    print the document on the paper;  
    signal(S);  
}
```


2.4.4 信号量的应用

习题课





信号量应用解题思路

- ❑ Step 1: 分析问题，确定进程数、进程活动以及各进程的活
动；
- ❑ Step 2: 确定进程间的关系：互斥、同步、资源管控问题；
- ❑ Step 3: 确定信号量个数；
- ❑ Step 4: 设置信号量的初值；
- ❑ Step 5: 将P、V操作放到合适的位置。



习题1

在一个盒子里，混装了数量相等的黑白围棋子。现在用自动分拣系统把黑子、白子分开，设分拣系统有两个进程P1和P2，其中P1拣白子，P2拣黑子。规定当一个进程拣了一子后，必须让另一个进程去拣。用信号量和wait、signal操作协调两进程的活动。

灵魂3问：

● 互斥问题or同步问题or资源管控问题？

同步

● 需要几个信号量？

2个，需要循环执行

● 初值如何设置？

分别设置为0和1



习题2

某控制系统中，数据采集进程负责把采集到的数据放到一缓冲区中；分析进程负责把数据从缓冲区中取出进行分析，试用信号量实现两者之间的同步。

灵魂3问：

● 互斥问题or同步问题or资源管控问题？

同步

● 需要几个信号量？

2个，需要循环执行

● 初值如何设置？

分别设置为0和1



习题3

图书馆规定，每位进入图书馆的读者要在登记表上登记，退出时要在登记表上注销。

(1) 用信号量实现读者之间的互斥登记和注销；

(2) 图书馆共有100个座位，当图书馆中没有空座位时，后到的读者在图书馆要等待（阻塞）。

灵魂3问：

互斥和资源管控

● 互斥问题or同步问题or资源管控问题？

● 需要几个信号量？

2个，1个互斥，1个资源管控

● 初值如何设置？

分别设置为1和100



习题4

一家四人父、母、儿子、女儿围桌而坐；桌上有一个水果盘。

- 当水果盘空时，父亲可以放香蕉，或母亲可以放苹果
- 盘中已有水果时，就不能放，父母等待。
- 当盘中有香蕉时，女儿可吃香蕉，否则，女儿等待；
- 当盘中有苹果时，儿子可吃，否则，儿子等待。



习题5

在公共汽车上，司机和售票员的活动分别是：

司机的活动： 启动车辆

正常运行

到站停车

售票员的活动： 关车门

售票

开车门

在汽车不断的到站，停车，行驶过程中，司机和售票员的活动有什么同步关系？用信号量和 P，V 操作实现。



习题6

有一个超市，最多可容纳 N 个人进入购物，当 N 个顾客满员时，后到的顾客在超市外等待；超市中只有一个收银员。可以把**顾客和收银员**看作**两类进程**，两类进程间存在**同步**关系。写出用P、V操作实现的两类进程的算法。

顾客活动：进入超市
购物
结账
离开

收银员活动：开始收银



● 2.4.5 管程机制（自学）

- 1. 管程的定义
- 2. 条件变量



WARNING

为了便于阅读和编写程序，从本节课开始将wait(S)替换为P(S)，将signal(S)替换为V(S)。